

TECHNISCHE UNIVERSITÄT
CHEMNITZ

AI-Augmented FAIRness Assessment of Research Software: A Hybrid Approach

Master Thesis

Submitted in Fulfilment of the
Requirements for the Academic Degree
M.Sc.

Dept. of Computer Science
Chair of Computer Engineering

Submitted by: Klea Haxhiu
Student ID: [your student ID]
Date: September 9, 2026

Supervising tutor: [your supervisor name]

Abstract

Keywords: FAIR, research software, LLM, documentation quality, FAIR-ness assessment

Contents

Contents **4**

List of Figures **8**

List of Tables **9**

List of Abbreviations **10**

1 Introduction **11**
1.1 Motivation 11
1.2 Problem Statement 12
1.3 Research Objective and Research Questions 12
1.4 Thesis Structure 13

2 State of the Art **14**
2.1 The FAIR Principles 14
2.2 FAIR for Research Software 15
2.3 Existing FAIRness Assessment Approaches 16
2.3.1 F-UJI 16
2.3.2 FAIR-IMPACT Metrics 16
2.3.3 FAIR-Aware 17
2.3.4 FAIRsharing 17
2.3.5 Summary 17
2.4 Software Documentation Quality 18
2.4.1 README Content and Structure 18
2.4.2 Beyond Accuracy: Multi-Dimensional Documentation Quality 18
2.4.3 Practitioner Perspective on Documentation Quality 18
2.4.4 Summary 19
2.5 LLM-Based Document Analysis 19
2.5.1 LLM-Based FAIR Assessment 19
2.5.2 LLMs for Software Engineering 20
2.5.3 LLM-Based Code Documentation Assessment 20
2.5.4 LLM-as-Judge 21
2.5.5 Summary 21
2.6 Research Gap 21

3 Methodology and System Design **23**
3.1 Overall Research Design 23
3.2 Dataset Collection 24
3.2.1 Research Software Definition 24
3.2.2 Collection Strategy 24
3.2.3 Filtering and Cleaning 24
3.2.4 Dataset Characteristics 24
3.3 Ground Truth Construction 25
3.3.1 Purpose and Scope 25
3.3.2 Repository Selection 25
3.3.3 Annotation Guidelines 26
3.3.4 Annotation Process 26
3.3.5 Inter-Annotator Agreement 26
3.4 Rule-Based Assessment 27
3.4.1 Overview 27
3.4.2 Indicators 27
3.4.3 Implementation 28
3.4.4 Output 28
3.5 LLM-Based Assessment 29
3.5.1 Overview 29
3.5.2 Documentation Quality Dimensions 29
3.5.3 Model Selection 30
3.5.4 Prompt Design 30
3.5.5 Implementation 30
3.5.6 External Link Policy 31
3.5.7 Output 31
3.6 Repository FAIRness Profile 32
3.7 Evaluation Strategy 32
3.7.1 Ground Truth Comparison 32
3.7.2 Inter-Annotator Agreement 33
3.7.3 Baseline Comparison 33
3.7.4 Qualitative Error Analysis 33
3.7.5 Confidence Calibration 33
3.7.6 README Length and Verbosity Bias 34

4 Implementation **35**
4.1 System Architecture and Overview 35
4.2 Repository Data Collection and Local Representation 37
4.3 Rule-Based Assessment Implementation 38
4.4 LLM-Based Assessment Implementation 39
4.5 Hybrid Profile Generation 40
4.6 Example Repository Profile 40
4.6.1 Rule-Based Indicators 40
4.6.2 LLM-Based Documentation Dimensions 41

4.6.3 Interpretation . . . 41
4.7 Implementation Summary . . . 41

5 Evaluation and Results . . . 43
5.1 Experimental Setup . . . 43
5.1.1 Dataset . . . 43
5.1.2 Evaluation Metrics . . . 43
5.1.3 Models and Tools . . . 44
5.1.4 Experimental Environment . . . 44
5.1.5 Evaluation Procedure . . . 44
5.2 Ground Truth Validation . . . 45
5.2.1 Annotation Process . . . 45
5.2.2 Inter-Annotator Agreement . . . 45
5.3 Rule-Based Assessment Results . . . 47
5.3.1 Full Dataset Results . . . 47
5.3.2 Validation Against Ground Truth . . . 48
5.4 LLM-Based Assessment Results . . . 49
5.4.1 LLM vs Primary Annotator . . . 49
5.4.2 LLM vs Second Annotator . . . 49
5.4.3 Pilot-Exclusion Sensitivity Analysis . . . 49
5.4.4 Systematic Scoring Patterns . . . 51
5.4.5 Comparison with Inter-Annotator Agreement . . . 51
5.4.6 Confidence Calibration . . . 52
5.4.7 README Length and Verbosity Bias . . . 53
5.5 Hybrid FAIRness Profile Analysis . . . 54
5.5.1 Score Distributions . . . 54
5.5.2 Relationship Between Rule-Based and LLM Scores . . . 54
5.5.3 Complementary Assessment Dimensions . . . 55
5.5.4 Profile Interpretation . . . 55
5.6 Baseline Comparison . . . 55
5.6.1 F-UJI Baseline Applicability . . . 55
5.6.2 howfairis Assessment Results . . . 56
5.6.3 Scope Comparison . . . 57
5.7 Summary of Findings . . . 58
5.7.1 Ground Truth Reliability . . . 58
5.7.2 Rule-Based Assessment . . . 58
5.7.3 LLM-Based Assessment . . . 58
5.7.4 Hybrid FAIRness Profile . . . 59
5.7.5 Baseline Comparison . . . 59
5.7.6 Summary . . . 59

6 Discussion . . . 60
6.1 RQ1: LLM-Based Assessment as a Complement to Rule-Based Assessment . . . 60

6.2 RQ2: Reliability of LLM-Based Assessment and Its Failure Modes . . 61
6.3 Comparison with Related Work 62
6.4 Implications 64
6.5 Limitations and Threats to Validity 65
6.6 Future Work 67

7 Conclusion **69**

Bibliography **71**

List of Figures

4.1 System architecture of the implemented hybrid FAIRness assessment pipeline. 36

List of Tables

4.1 Rule-based indicator scores for `astropy/astropy`. 41
4.2 LLM-based documentation scores for `astropy/astropy`. 42

5.1 Inter-annotator agreement for rule-based indicators and AI-based dimensions (n=30). 46
5.2 Rule-based indicator frequencies across the full dataset (n=188). . . . 47
5.3 Distribution of rule-based total scores across the full dataset (n=188). 48
5.4 Rule-based validation against manual ground truth (n=30). 48
5.5 LLM vs primary annotator agreement (n=30). 49
5.6 LLM vs second annotator agreement (n=30). 50
5.7 LLM agreement with the primary annotator on the full annotated corpus and the 20-repository pilot-exclusion subset. 50
5.8 Direction of LLM scoring disagreements relative to primary annotator (n=30). LLM lower = LLM score < human score; LLM higher = LLM score > human score. 51
5.9 Agreement summary: human-human vs LLM-human (macro-avg weighted κ, AI dimensions only). 52
5.10 LLM confidence calibration against the primary human annotations across all documentation dimensions. 52
5.11 Mean scores for rule-based and LLM-based components across the 30 ground truth repositories. 54
5.12 F-UJI assessment results compared with the proposed framework's rule-based and LLM documentation scores (n=5). 56
5.13 howfairis score distribution across the 30 ground truth repositories. . 57
5.14 Comparison of assessment scope across howfairis, the rule-based component, and the LLM-based component. 57

List of Abbreviations

FAIR Findable, Accessible, Interoperable, Reusable

FAIR4RS FAIR for Research Software

LLM Large Language Model

API Application Programming Interface

README Read Me

DOI Digital Object Identifier

1 Introduction

1.1 Motivation

The increasing importance of open science and reproducibility has led to a growing need for effective management and reuse of digital research outputs, including data, software, and computational workflows. The FAIR principles — Findable, Accessible, Interoperable, and Reusable — were introduced by Wilkinson et al. [11] to address these challenges by providing guidelines that enhance the discoverability and reuse of research artifacts. A central goal of the FAIR principles is machine-actionability — the ability of automated agents to discover, evaluate, and act on digital objects without prior knowledge of them.

While initially developed for research data, the FAIR principles are equally applicable to research software, which plays a central role in modern scientific processes. The FAIR for Research Software (FAIR4RS) principles [1] extend and adapt the original framework to address the specific characteristics of software, such as executability, versioning, and dependency management. Ensuring that software is FAIR is essential for enabling reproducibility, transparency, and collaboration.

However, in practice, many research software repositories lack essential elements including structured metadata, clear licensing information, persistent identifiers, and comprehensive documentation. Assessing the FAIRness of research software at scale remains a challenging task, particularly for indicators that depend on the quality and completeness of software documentation [2].

Existing FAIRness assessment approaches primarily rely on rule-based checks that evaluate the presence of metadata files, licenses, persistent identifiers, and other objectively measurable indicators [4]. While these approaches provide valuable automated assessments, they often struggle to evaluate qualitative aspects such as installation guidance, usage instructions, and documentation quality — factors that directly influence software reuse and reproducibility [2].

Recent advances in Large Language Models (LLMs) offer new opportunities for analyzing natural language documentation and supporting FAIRness assessment beyond traditional rule-based methods. By interpreting README files and other documentation artifacts, LLMs have the potential to evaluate aspects that are difficult to capture using deterministic rules alone.

This thesis investigates how AI-based documentation assessment can complement rule-based FAIRness evaluation for research software through a hybrid assessment approach that combines automated file-based checks with LLM-supported analysis of repository documentation. The goal is to investigate whether LLM-based assessment

can capture qualitative FAIR-related documentation characteristics that are not addressed by rule-based methods alone.

1.2 Problem Statement

Current methods for evaluating the FAIRness of research software repositories are limited in their ability to scale and to assess unstructured information. Rule-based approaches, such as those implemented in F-UJI [4], can effectively detect explicit features such as the presence of a license file or structured metadata, but are insufficient for evaluating qualitative aspects such as documentation clarity, completeness, and usability. As noted by Chue Hong et al. [2], automated tools can verify the existence of metadata but cannot reliably determine its quality, relevance, or usefulness.

Research on Mining Software Repositories has shown that README files are the primary documentation artifact for GitHub repositories and contain rich information about a project's purpose, usage, and quality [8]. However, the systematic integration of documentation-based information into automated FAIRness assessment workflows remains limited. Rule-based tools can verify whether a README file exists, but cannot determine whether it adequately describes installation procedures, usage examples, or software dependencies. Treude et al. [10] further demonstrate that documentation quality goes beyond mere presence — dimensions such as clarity, understandability, and structure require qualitative assessment that rule-based approaches cannot provide.

The importance of documentation quality extends beyond academic frameworks. Imani et al. [6] found that over 91% of industry practitioners rely on OSS documentation for software adoption decisions, and that the top reported challenge is not the absence of documentation but the difficulty of finding relevant information within it. This practitioner evidence reinforces the need for assessment methods that evaluate not just the presence but the quality and completeness of documentation content.

Consequently, there is a need to investigate whether a hybrid approach that combines rule-based structural checks with LLM-based qualitative assessment can extend automated FAIRness evaluation to documentation characteristics that are not captured by rule-based methods alone.

1.3 Research Objective and Research Questions

The primary objective of this thesis is to design and implement a prototype system for the automated assessment of FAIRness in research software repositories hosted on GitHub. The system combines rule-based indicators derived from the FAIR4RS principles [1] with an LLM-based documentation assessment component and evaluates the extent to which the latter complements rule-based assessment by capturing qualitative documentation characteristics. The following research questions guide

the investigation:

RQ1 To what extent can LLM-based assessment complement rule-based FAIRness assessment by capturing qualitative documentation characteristics of research software repositories?

RQ2 How reliably can an LLM assess qualitative documentation dimensions of research software repositories, and what failure modes affect its assessments?

The system is implemented as a modular Python prototype focused on GitHub-hosted repositories. Extensibility to other platforms such as Zenodo or Codeberg is considered out of scope for this thesis but is noted as a direction for future work.

1.4 Thesis Structure

The remainder of this thesis is organized as follows.

Chapter 2 reviews the relevant literature, covering the FAIR principles and their adaptation for research software, existing automated FAIRness assessment tools, research on software documentation quality, and recent advances in LLM-based document analysis.

Chapter 3 describes the proposed hybrid assessment approach, including the dataset collection methodology, the rule-based component, the AI-based documentation assessment component, and the scoring and output design.

Chapter 4 describes the implementation of the proposed framework, including the repository processing pipeline, rule-based assessment component, LLM integration, and generation of hybrid FAIRness profiles.

Chapter 5 presents the evaluation strategy and results, including ground-truth validation, quantitative comparison against manual annotations, analysis of LLM scoring behaviour, and comparison with existing FAIRness assessment tools.

Chapter 6 discusses the findings in relation to the research questions, considers the implications and limitations of the approach, and outlines directions for future work.

Chapter 7 concludes the thesis by summarising the main contributions and findings.

2 State of the Art

2.1 The FAIR Principles

The FAIR principles were introduced by Wilkinson et al. [11] as a minimal, domain-independent framework for improving the findability, accessibility, interoperability, and reusability of digital research objects. The principles were designed to support both human and machine interaction with research outputs, with machine-actionability as a central goal — the ability of automated agents to discover, evaluate, and act on digital objects without prior knowledge of them.

The four FAIR principles are defined as follows.

A digital object is **Findable** if it is assigned a globally unique and persistent identifier, described with rich metadata, and indexed in a searchable resource.

It is **Accessible** if it can be retrieved via an open and standardized protocol, and if its metadata remains accessible even when the object itself is no longer available.

Interoperability requires that the object uses formal knowledge representation languages, FAIR vocabularies, and qualified references to other objects.

Finally, an object is **Reusable** if it has rich attributes, a clear license, detailed provenance, and meets community standards.

An important characteristic of the FAIR principles is that they are modular and independent — they can be adopted incrementally and in any combination. Furthermore, FAIRness is treated as a continuum rather than a binary property: a digital object can be more or less FAIR depending on how completely it satisfies each principle. This continuum directly motivates the development of automated assessment tools capable of measuring FAIRness at scale.

The FAIR principles have become one of the most influential frameworks in research data management and open science. They have been adopted by funding agencies, research infrastructures, publishers, and scientific communities worldwide as a reference framework for improving the management and reuse of digital research outputs. Their widespread adoption has also motivated the development of FAIR assessment tools, metrics, and maturity indicators.

Despite their influence, the FAIR principles intentionally do not prescribe specific technologies, implementation strategies, or minimum compliance requirements. As a result, determining the degree of FAIRness of a digital object can be challenging and often requires interpretation. This has led to the development of automated assessment frameworks and community-specific implementations that translate the principles into measurable indicators.

While the FAIR principles were originally developed for research data, Wilkinson

et al. explicitly noted their applicability to other scholarly digital objects including software, workflows, and algorithms. Consequently, while the FAIR principles provide a general conceptual framework, additional guidance is required to address software-specific characteristics and to enable meaningful FAIRness assessment of research software repositories.

2.2 FAIR for Research Software

Although the FAIR principles were originally formulated for research data, software has become an equally important research output in modern science. Research software differs from data in several ways, including its executable nature, dependency structure, versioning requirements, and ongoing maintenance needs. As a result, direct application of the original FAIR principles to software can be challenging.

To address these challenges, the FAIR for Research Software (FAIR4RS) principles were introduced by Barker et al. [1]. The FAIR4RS principles were developed through a large community consultation process involving approximately 500 contributors from more than 110 organisations, reflecting broad international consensus on the need for software-specific FAIR guidance. The principles adapt the original framework to account for the unique characteristics of software, introducing new sub-principles where necessary.

Under FAIR4RS, software is **Findable** if it is assigned globally unique and persistent identifiers, with distinct identifiers for different versions and components — an extension not present in the original FAIR principles. Software is **Accessible** if it can be retrieved via standardised protocols and its metadata remains available even if the software itself becomes unavailable. **Interoperability** in FAIR4RS extends beyond metadata to include API interactions and references to related software objects. Finally, software is **Reusable** if it is executable, understandable, modifiable, and integrable into other systems, with clear licensing, provenance, and community standards.

Research software plays a critical role in enabling reproducible science, and FAIR4RS therefore positions software discoverability, accessibility, and reusability as essential requirements for transparent and reproducible research workflows.

A key insight of FAIR4RS is that FAIRness for software should begin during development rather than only at publication time. Unlike data, software may be used directly from its development environment, making early adoption of FAIR practices essential for reproducibility and reuse. The principles have received growing institutional support from organisations such as ELIXIR, ARDC, the Netherlands eScience Center, and ZB MED.

Despite this progress, FAIR4RS defines high-level principles without prescribing concrete automated assessment methods. Many FAIR4RS indicators rely on metadata and structural information, while qualitative aspects such as documentation clarity, installation guidance, and usage examples remain difficult to evaluate automatically. This gap between the principles and their practical assessment motivates

the development of automated tools capable of evaluating both structured and unstructured aspects of research software repositories.

2.3 Existing FAIRness Assessment Approaches

Several tools and frameworks have been developed to automate the assessment of FAIRness for digital research objects. This section reviews the most relevant approaches, highlighting their capabilities and limitations in the context of research software assessment.

2.3.1 F-UJI

F-UJI is an automated web-based FAIR assessment tool developed within the FAIRsFAIR initiative [4]. It evaluates the FAIRness of digital research objects by performing machine-actionable checks against predefined metrics, producing quantitative scores across the four FAIR dimensions. F-UJI accepts persistent identifiers and metadata endpoints as input and evaluates indicators such as metadata presence, persistent identifiers, licensing, semantic vocabularies, provenance information, and accessibility protocols.

F-UJI treats FAIRness as a spectrum rather than a binary property, providing detailed metric-level reporting and evidence traces. It demonstrates that many aspects of FAIR compliance can be assessed automatically through metadata checks. However, F-UJI is primarily metadata-centric and was originally designed for FAIR digital objects and datasets rather than research software repositories specifically. It was not designed for direct GitHub repository assessment and expects persistent identifiers and structured metadata records as input, which many repositories in practice do not provide.

Critically, F-UJI evaluates the existence and machine-readability of metadata rather than its semantic quality or usefulness. Documentation quality, understandability, and usability are not assessed, creating a gap between structural FAIR compliance and meaningful software reusability.

2.3.2 FAIR-IMPACT Metrics

The FAIR-IMPACT initiative extended existing FAIR assessment approaches by defining 17 concrete, measurable FAIR Research Software Metrics (FRSM-01 to FRSM-17) for automated assessment of research software against the FAIR4RS principles [2]. These metrics operationalize FAIR4RS into practical automated checks, covering findability, accessibility, interoperability, and reusability indicators at the code, project, and repository level.

The metrics use three compliance levels — Essential, Important, and Useful — and are designed to be domain-agnostic while adaptable to disciplinary contexts. Planned integration into F-UJI demonstrates their practical application potential.

However, the FAIR-IMPACT metrics explicitly acknowledge that automated tools can verify the existence of metadata but cannot reliably determine its quality, relevance, or correctness. Qualitative aspects such as documentation clarity and usability remain outside the scope of automated rule-based assessment.

2.3.3 FAIR-Aware

In contrast to automated assessment tools, FAIR-Aware is a human-centred self-assessment tool developed within the FAIRsFAIR initiative that focuses on increasing FAIR literacy among researchers. It uses a questionnaire-based approach to help researchers reflect on how FAIR practices can be integrated into their workflows, providing guidance and recommendations rather than automated compliance metrics.

FAIR-Aware demonstrates that FAIRness assessment is not limited to verifying machine-readable metadata. Many FAIR considerations require researchers to interpret community standards, evaluate reuse conditions, and make contextual decisions that depend on domain knowledge and qualitative judgment. This highlights the importance of qualitative reasoning in FAIR evaluation — a dimension that automated rule-based tools cannot fully address.

2.3.4 FAIRsharing

FAIRsharing is a curated registry of FAIR-related standards, repositories, databases, policies, and metadata schemas. It provides centralised infrastructure to support the discovery, adoption, and interconnection of community standards and FAIR-enabling resources across research domains. FAIRsharing demonstrates that FAIRness is deeply connected to the existence and adoption of shared standards and controlled vocabularies — automated FAIR assessment systems rely heavily on this ecosystem infrastructure to validate interoperability and machine-readability. FAIRsharing is particularly relevant for research software because it supports the discovery of community standards, metadata schemas, and domain-specific resources that contribute to software interoperability and reuse.

2.3.5 Summary

Existing FAIRness assessment tools provide valuable automated evaluation capabilities and have significantly advanced the practical application of FAIR principles. However, their primary focus remains on structured metadata and objectively measurable indicators, leaving qualitative aspects of software documentation largely unexplored.

2.4 Software Documentation Quality

While existing FAIRness assessment tools focus on structured metadata and file presence, research on software documentation quality highlights a complementary dimension of software reusability that these tools do not address. README files, in particular, serve as the primary documentation artifact for GitHub repositories and play a central role in how users discover, understand, and reuse software.

2.4.1 README Content and Structure

Prana et al. [8] conducted the first large-scale empirical study of GitHub README files, categorising their content into eight types: What, Why, How, When, Who, References, Contribution, and Other. Their study of 393 repositories found that while installation and usage instructions (How) are present in the majority of files, purpose and motivation sections (Why) appear in only 25.7% of repositories, and project status information (When) in only 21.4%.

This systematic absence of purpose and status information is not merely a documentation style issue — it reflects a gap in communicating software maturity and intent that directly maps to FAIR4RS reusability requirements. Rule-based tools can detect whether a README file exists, but cannot determine whether it contains the information a user needs to evaluate or reuse the software.

2.4.2 Beyond Accuracy: Multi-Dimensional Documentation Quality

Treude et al. [10] proposed a ten-dimensional framework for assessing software documentation quality, adapted from the information quality literature. The framework deliberately goes beyond accuracy and completeness — the dominant focus of prior work — to cover dimensions such as clarity, understandability, structure, readability, and conciseness.

A key theoretical contribution of this framework is the deliberate exclusion of accuracy and completeness as primary dimensions, since these require external reference points. This means that even a complete README — one that passes rule-based checks — can still score poorly on clarity or understandability. The framework directly supports the argument that rule-based FAIRness assessment is insufficient on its own, and that dimensions such as clarity and understandability require a qualitative assessment approach.

2.4.3 Practitioner Perspective on Documentation Quality

Imani et al. [6] provided empirical evidence from an industry perspective, finding that over 91% of practitioners rely on OSS documentation for software adoption decisions. Their study identified that the top reported challenge is not the absence of documentation but the difficulty of finding relevant information within it. Nine

adoption criteria emerged from practitioner interviews, including ease of use, compatibility, functionality, and presence of usage examples — criteria that closely map to FAIR4RS reusability indicators.

This practitioner evidence reinforces the need for assessment methods that evaluate not just the presence but the quality and completeness of documentation content.

These findings suggest that structured rubrics and explicit scoring criteria are important design considerations for LLM-based assessment systems, particularly when consistency and reliability are required.

2.4.4 Summary

Research on software documentation quality consistently demonstrates that documentation presence alone is insufficient to ensure software reusability. The content, clarity, and completeness of documentation are critical factors that current automated FAIRness assessment tools do not evaluate. This creates a clear motivation for approaches that combine traditional rule-based indicators with qualitative assessment of documentation content.

2.5 LLM-Based Document Analysis

Recent advances in large language models (LLMs) have opened new possibilities for automated analysis of natural language artifacts in software repositories. This section reviews work on LLM capabilities for software engineering tasks, code documentation generation, and automated evaluation — providing the technical foundation for the AI-based component of this thesis.

2.5.1 LLM-Based FAIR Assessment

A closely related contribution is Fair-Way [9], an open-source framework that applies Large Language Models (LLMs) to automate FAIRness assessment of research datasets. Unlike traditional rule-based tools, Fair-Way uses LLMs to evaluate FAIRsFAIR Data Object Assessment Metrics (DOAM), aiming to improve flexibility across heterogeneous metadata formats and domains.

Fair-Way follows a divide-and-conquer strategy, decomposing the assessment into fine-grained tasks while splitting large metadata files into manageable chunks. The system employs prompt engineering techniques including structured JSON output, task decomposition, chain-of-thought prompting, and few-shot examples to improve assessment reliability and reproducibility. The authors evaluate multiple LLMs, reporting GPT-4o as the best-performing model, and compare Fair-Way against F-UJI on three datasets from different scientific domains. Their results demonstrate performance comparable to, and in several assessment metrics exceeding, the rule-based baseline.

Despite these promising results, Fair-Way differs substantially from the approach proposed in this thesis. First, it focuses on research datasets and FAIRsFAIR DOAM metrics, whereas this work targets research software repositories and FAIR4RS-inspired documentation indicators. Second, Fair-Way relies on a purely LLM-based assessment pipeline, while the authors explicitly identify hybrid approaches combining LLMs with explicit rule-based verification as an important direction for future work. Finally, Fair-Way evaluates dataset metadata, whereas this thesis combines rule-based repository analysis with LLM-based assessment of GitHub repository documentation to produce a hybrid FAIRness evaluation for research software.

These differences highlight an important gap in the current literature. While Fair-Way demonstrates the potential of LLMs for automated FAIR assessment, the literature reviewed in this thesis does not identify an approach that combines rule-based repository analysis with LLM-based documentation assessment specifically for research software repositories.

2.5.2 LLMs for Software Engineering

Hou et al. [5] conducted a systematic literature review of LLM applications across the software engineering lifecycle, finding that LLMs demonstrate strong performance on tasks involving natural language artifacts such as documentation, code summarization, and explanation generation. A central argument of the review is that many software engineering tasks involving documentation, explanation, and comprehension are fundamentally language understanding problems that benefit from the semantic reasoning capabilities of LLMs.

The review also identifies a consistent recommendation across studies: hybrid approaches combining LLMs with traditional methods are preferable for reliability-critical applications, as LLM limitations such as hallucination, prompt sensitivity, and inconsistent outputs warrant complementary rule-based checks. This directly supports the hybrid architecture proposed in this thesis.

2.5.3 LLM-Based Code Documentation Assessment

Dvivedi et al. [3] conducted a comparative study of multiple LLMs for code documentation generation, evaluating outputs on six quality dimensions including accuracy, completeness, relevance, understandability, and readability. Their study found that closed-source models consistently outperform open-source alternatives, and that file-level documentation — analogous to README files — is consistently the most challenging documentation type for LLMs to assess reliably.

The study also validated that checklist-based evaluation rubrics reduce inter-rater subjectivity, achieving only 11.67% disagreement across raters. This finding supports the use of structured rubrics in LLM-based scoring — the approach adopted in this thesis for evaluating README documentation quality against FAIR4RS indicators.

2.5.4 LLM-as-Judge

Zheng et al. [12] conducted the first systematic empirical study of the LLM-as-a-judge paradigm, evaluating the use of strong LLMs as automated evaluators of open-ended responses. They found that GPT-4 as judge achieves over 80% agreement with human raters, approaching human–human agreement levels in the evaluated settings. Their results demonstrate that strong LLMs can achieve substantial agreement with human raters in some open-ended evaluation settings, while also showing that reliability depends on the evaluation setup and model used.

The study also characterised key biases of LLM judges, including position bias, verbosity bias, and self-enhancement bias. For single answer grading — the scenario most relevant to README quality assessment — the study found that absolute scores are less stable than relative comparisons, and that providing structured rubrics with explicit criteria substantially improves scoring reliability. These findings motivate the use of explicit scoring criteria and the investigation of potential scoring biases in the LLM-based assessment developed in this thesis.

2.5.5 Summary

LLMs offer strong capabilities for analyzing natural language documentation and assessing qualitative aspects of software artifacts that rule-based approaches cannot evaluate. However, existing work also highlights consistent limitations including output instability, prompt sensitivity, and domain-specific challenges. These limitations motivate explicit evaluation of LLM reliability, scoring consistency, and failure modes when LLMs are used for qualitative assessment.

2.6 Research Gap

The literature reviewed in this chapter reveals a clear and consistent gap in existing approaches to FAIRness assessment of research software.

Automated FAIRness assessment tools such as F-UJI and the FAIR-IMPACT metrics have made significant progress in evaluating structured, machine-readable aspects of research software repositories, including metadata presence, persistent identifiers, licensing, and dependency specifications. However, these tools predominantly rely on rule-based checks that detect the presence or absence of explicit repository features. They do not analyse the quality or content of unstructured documentation such as README files, which research has shown to be critical indicators of software reusability and usability [8, 10].

Research on software documentation quality demonstrates that dimensions such as clarity, understandability, and completeness of installation and usage instructions are important for software adoption and reuse [6], yet these dimensions are outside the scope of existing automated FAIRness assessment approaches. The FAIR-IMPACT metrics explicitly acknowledge that automated tools can verify the existence of metadata but cannot reliably determine its quality or usefulness [2].

Recent advances in large language models offer a promising approach to filling this gap. LLMs have demonstrated strong capabilities for analyzing natural language documentation and evaluating qualitative aspects of software artifacts [5, 3]. The LLM-as-a-judge paradigm has demonstrated the potential of LLMs to support automated evaluation of open-ended outputs, although its reliability depends on the model, task, and evaluation setup [12]. Despite this potential, the integration of LLMs into FAIRness assessment workflows remains largely unexplored. Within the literature reviewed in this thesis, this combination of rule-based FAIR indicators and LLM-based documentation quality assessment for research software repositories remains underexplored.

This thesis addresses this gap by designing, implementing, and evaluating a hybrid assessment approach that combines automated rule-based checks with LLM-based analysis of README documentation against FAIR4RS reusability and findability indicators. The approach aims to complement existing FAIRness assessment tools by providing qualitative evaluation of documentation dimensions that rule-based methods cannot assess, while also investigating the reliability and limitations of LLM-based assessment.

3 Methodology and System Design

3.1 Overall Research Design

This thesis proposes and evaluates a hybrid approach to automated FAIRness assessment of research software repositories. The approach combines two complementary assessment components: a rule-based component that evaluates the presence of structured metadata and repository files against indicators derived from the FAIR4RS principles [1], and an AI-based component that evaluates the quality and completeness of README documentation using a Large Language Model. Together, these components provide complementary evidence that forms the basis for the repository FAIRness assessment.

The motivation for a hybrid design stems from the limitations of existing approaches. Rule-based tools such as F-UJI [4] and howfairis assess the presence or absence of structured metadata — license files, persistent identifiers, citation files, and dependency specifications — but cannot evaluate qualitative characteristics of documentation, such as whether installation instructions, usage guidance, examples, or contribution information are sufficiently complete for software reuse. The rule-based component produces deterministic and reproducible assessments, while the LLM-based component captures qualitative aspects of documentation that are difficult to express through fixed rules.

The research design follows four main phases. First, a dataset of 188 research software repositories is collected from GitHub using a systematic topic-based search strategy. Second, a ground truth set of 30 repositories is manually annotated against a structured rubric derived from FAIR4RS principles, providing a reference for evaluating the accuracy of both assessment components. Third, the rule-based and LLM-based components are implemented and validated against the ground-truth annotations. The rule-based component is subsequently applied to the full 188-repository dataset, while the LLM-based evaluation is conducted on the 30-repository annotated corpus. Fourth, the outputs of both components are evaluated to assess agreement between automated assessments and manual annotations, and to compare the proposed approach with existing baseline tools.

The following sections describe each component of the methodology in detail.

3.2 Dataset Collection

3.2.1 Research Software Definition

Research software is defined in this work as software developed to support, enable, or conduct scientific research, including data analysis tools, simulation software, computational pipelines, scientific frameworks, and domain-specific research libraries [1]. Repositories were considered research software if they demonstrated a clear scientific or research context through their descriptions, repository topics, README documentation, associated publications, or the affiliation of the owning organisation.

3.2.2 Collection Strategy

To obtain a diverse dataset, repositories were collected from ten research domains: bioinformatics, computational biology, astronomy, neuroscience, climate science, scientific computing, research software, cheminformatics, computational physics, and reproducible research. Each domain was queried using the GitHub Search API with the following pattern:

```
topic:<domain> stars:>20 archived:false
```

A minimum threshold of 20 GitHub stars was applied to filter out inactive or personal experimental repositories. Archived repositories were excluded as they are no longer actively maintained. The collection was capped at 20 repositories per topic to ensure balanced domain representation, yielding an initial dataset of 198 repositories.

3.2.3 Filtering and Cleaning

Following collection, repositories were automatically filtered according to predefined inclusion criteria. Forks, archived repositories, repositories whose most recent push occurred before 1 January 2023, and repositories whose names or descriptions contained predefined keywords — such as *awesome*, *tutorial*, *course*, *template*, *workshop*, *lecture*, *homework*, *assignment*, and *cheatsheet* — were excluded, as these typically represent curated lists, educational materials, or project templates rather than executable research software. These filtering steps reduced the initial dataset from 198 to 188 repositories.

3.2.4 Dataset Characteristics

The final dataset contains 188 research software repositories spanning 10 research domains and multiple programming language ecosystems. Python is the most common language (86 repositories, 46%), followed by C++ (29, 15%) and Jupyter Notebook (18, 10%). Additional languages include Julia, R, Rust, Java, Fortran, Go, and

JavaScript, providing diversity beyond a single programming ecosystem. All repositories were publicly accessible on GitHub and had been actively maintained at the time of collection, whose latest repository activity occurred on or after 1 January 2023.

For each repository, the following data were collected via the GitHub API: repository metadata (name, owner, description, primary language, star count, topics, and last push date), README file content, and the presence of FAIR-related repository files including license files, citation files, dependency specifications, structured metadata files, and contribution guidelines.

3.3 Ground Truth Construction

3.3.1 Purpose and Scope

To evaluate the accuracy of both the rule-based and LLM-based assessment components, a manually annotated ground-truth corpus of 30 research software repositories was constructed. Of these, 23 repositories are also included in the final 188-repository dataset, while seven originate from the earlier annotation corpus and are not members of the final collected dataset. Manual annotations provide reference scores against which the automated assessments can be compared, enabling the computation of agreement metrics and the identification of systematic scoring errors.

3.3.2 Repository Selection

The annotated corpus was constructed in two stages. An initial set of 10 repositories was annotated during the pilot phase of system development and was used to refine the annotation guidelines and assessment procedure. The corpus was subsequently expanded to 30 repositories to increase diversity across research domains and programming-language ecosystems, including C++, Java, MATLAB, Fortran, Julia, Go, Rust, and Groovy in addition to Python.

The final annotated corpus therefore contains both repositories from the initial pilot and repositories added during the subsequent expansion. Of the 30 annotated repositories, 23 are also represented in the final 188-repository dataset. The remaining seven were retained from the earlier annotation corpus but are not members of the final 188-repository collection.

Because the original 10 pilot repositories contributed to rubric development and prompt refinement, evaluation on the full 30-repository corpus is not treated as a fully independent test of the LLM-based component. Chapter 5 therefore additionally reports a pilot-exclusion sensitivity analysis in which these repositories are excluded from the LLM agreement calculation, leaving 20 repositories that were not used during the original pilot phase.

3.3.3 Annotation Guidelines

Annotations were performed using a structured rubric developed specifically for this thesis, covering both the rule-based indicators (F1, F2, A1, I1, R1, R2) and the AI-based documentation quality dimensions (D1–D6). The rubric defines precise scoring criteria for each indicator and dimension, including edge case handling for non-standard license filenames, alternative citation file formats, and the treatment of external documentation links.

The annotation guidelines were developed iteratively during the pilot phase and refined through validation against automated scorer outputs. The final version, v1.4, clarified the distinction between structured citation metadata (F2) and explicit citation instructions (R2), and resolved ambiguities identified during ground truth annotation. All 30 repositories were annotated against the finalised v1.4 guidelines to ensure consistency.

3.3.4 Annotation Process

Each repository was annotated by the thesis author by examining the repository root file structure and README content. Rule-based indicators (F1–R2) were scored by checking for the presence of specific files and patterns in the README text. AI-based dimensions (D1–D6) were scored on a 0–2 scale by reading the README and assessing the clarity and completeness of installation instructions, usage guidance, purpose description, dependency information, examples, and contribution information.

Each annotation record includes the assigned score together with evidence notes justifying the decision. For the AI-based dimensions, annotations additionally record the annotator's confidence level (HIGH, MEDIUM, or LOW), reflecting the degree of ambiguity encountered when applying the rubric criteria.

3.3.5 Inter-Annotator Agreement

To assess annotation reliability, a second annotator was provided with the finalised v1.4 annotation guidelines, the list of 30 repositories with GitHub links, pre-fetched README files, FAIR-related file presence summaries, and blank annotation forms. The annotation package was designed to enable fully independent annotation without requiring access to the primary annotations or the GitHub repositories directly.

Upon completion of the independent annotation process, inter-annotator agreement was quantified using Cohen's weighted Kappa for the AI-based dimensions (D1–D6), where partial agreement is meaningful, and Cohen's unweighted Kappa for the binary rule-based indicators (F1–R2). Exact agreement was additionally reported to aid interpretation in cases where Kappa was undefined because of a lack of class variation.

Disagreements and anomalous results were subsequently reviewed against the final annotation guidelines and the collected repository evidence. This validation

identified, among other cases, an inconsistency between the file-detection schema originally used for the ground-truth collection and the expanded schema used for the full dataset. The ground-truth file inventories and affected primary annotations were corrected where the repository evidence unambiguously supported the correction. The final agreement results are reported in Chapter 5.

3.4 Rule-Based Assessment

3.4.1 Overview

The rule-based assessment component evaluates each repository against six binary indicators derived from the FAIR4RS principles [1] and the FAIR-IMPACT research software metrics [2]. Each indicator checks for the presence of repository files or textual patterns that provide evidence for selected FAIR4RS indicators. The component produces a score of 0 or 1 per indicator, with a maximum total score of 6.

3.4.2 Indicators

The six rule-based indicators are defined as follows:

F1 — Persistent Identifier Score 1 if evidence of a persistent identifier (e.g., DOI, Zenodo, or Software Heritage identifier) is detected in the repository documentation or metadata. Detected patterns include doi.org URLs, Zenodo badges, Software Heritage identifiers, and DOI badge URL formats.

F2 — Structured Metadata Score 1 if a structured metadata file is present in the repository root. Accepted metadata files are `CITATION.cff` and `codemeta.json`, both of which provide machine-readable software metadata.

A1 — License Score 1 if licensing information is explicitly declared for the repository. This condition is satisfied if a recognized license file is present in the repository root, if a license is declared in the GitHub repository metadata, or if the README explicitly states the applicable license. Accepted license filenames include `LICENSE`, `LICENSE.md`, `LICENSE.txt`, `LICENSE.rst`, `COPYING`, and `COPYING.txt`.

I1 — Dependency Specification Score 1 if a dependency specification file is present in the repository root. Accepted files cover multiple language ecosystems, including `requirements.txt`, `pyproject.toml`, `environment.yml`, `setup.py`, `Cargo.toml`, `go.mod`, `pom.xml`, `Project.toml`, and `package.json`.

R1 — README Present Score 1 if a README file is present in the repository root. Accepted filenames include README.md, README.rst, README.txt, and README.

R2 — Citation Information Score 1 if explicit citation instructions are present in the repository. This includes dedicated citation files (CITATION.md, CITATION.bib, CITATION.txt, CITATION, CITATION.cff) and explicit citation instructions in the README text, such as phrases containing "please cite", "how to cite", "if you use … cite", or "we would appreciate citations".

3.4.3 Implementation

Repository data were collected in a preprocessing step and stored locally as JSON files. The scorer operates exclusively on these locally stored artefacts rather than querying the GitHub API during scoring, ensuring deterministic and reproducible execution. For each repository, the scorer reads the FAIR file presence record (`fair_file_checks.json`), the repository metadata (`metadata.json`), and the README file. File presence checks are performed against the stored JSON record, while selected indicators additionally inspect repository metadata and textual evidence. In particular, F1 searches for persistent identifier patterns, A1 checks repository metadata and explicit license statements in the README, and R2 searches for explicit citation instructions in the README text.

The scorer was validated against the 30 manually annotated ground truth repositories. Initial validation revealed discrepancies in several indicators. Detection logic for F1 and R2 was refined to cover additional DOI representations, citation file formats, and citation instruction phrasings identified during manual validation.

Accordingly, the rule-based validation against this corpus is interpreted primarily as a consistency check between the finalised rubric, repository evidence, and deterministic implementation rather than as a fully independent held-out evaluation.

A subsequent consistency check also identified schema drift between the collector originally used for the ground-truth repositories and the collector used for the full dataset. The earlier collector did not check several language-specific dependency and build files, including `Cargo.toml`, `go.mod`, `pom.xml`, `Project.toml`, and `build.gradle`. The ground-truth file inventories were therefore aligned with the expanded schema used for the full dataset, and affected annotations were corrected where the repository evidence unambiguously satisfied the final rubric. Additional manual annotation inconsistencies identified during validation were likewise corrected against the finalised guidelines. The final validation results, including precision, recall, and F1-score per indicator, are reported in Chapter 5.

3.4.4 Output

The rule-based scorer produces a CSV file containing one row per repository with individual indicator scores and the aggregated rule-based score. The scorer was subsequently applied to all 188 repositories in the dataset, producing the rule-based assessment results used in the evaluation presented in Chapter 5.

3.5 LLM-Based Assessment

3.5.1 Overview

The LLM-based assessment component evaluates the quality and completeness of repository README documentation against six documentation quality dimensions derived from FAIR4RS reusability indicators [1] and the software documentation quality framework proposed by Treude et al. [10]. Unlike the rule-based component, which checks for the presence of files and patterns, the LLM-based component assesses the content and clarity of documentation that a potential user would encounter when attempting to reuse the software.

3.5.2 Documentation Quality Dimensions

Six documentation quality dimensions are evaluated, each scored on a 0–2 scale:

D1 — Installation Clarity Assesses whether a new user can install the software from the README alone. Score 2 if complete and actionable installation steps are provided; score 1 if installation information exists but is incomplete or links externally; score 0 if no installation instructions are present.

D2 — Usage Clarity Assesses whether a new user can understand how to run or use the software from the README. Score 2 if a concrete command, code snippet, or executable example is present; score 1 if external documentation is clearly linked but no inline example appears; score 0 if no usage information is present.

D3 — Purpose Description Assesses whether the README clearly explains what the software does and its intended domain or audience. Score 2 if the purpose is clearly described; score 1 if a description is present but vague or incomplete; score 0 if no description is present.

D4 — Dependency Description Assesses whether the README describes the environment or dependencies required to use the software. Score 2 if dependencies are explicitly listed in human-readable form; score 1 if dependency information is only partially provided; score 0 if no dependency information is present.

D5 — Examples and Tutorials Assesses whether the repository provides concrete examples that support reuse. Score 2 if the README contains a meaningful usage example demonstrating software use. References to external example galleries or tutorials support score 1 unless their content is included in the supplied README evidence. Score 0 if no examples or tutorials are mentioned.

D6 — Contribution Information Assesses whether the README explains how to contribute to or get support for the software. Score 2 if contribution guidance is provided together with at least one community or support channel; score 1 if contribution or support information is only partially present; score 0 if neither is available.

3.5.3 Model Selection

The open-source Llama 3.1 8B model was selected because it can be executed locally through Ollama, ensuring reproducibility and eliminating dependence on commercial APIs. Running the model locally also avoids API usage costs and enables repeated experiments under identical inference settings. Temperature was set to 0 to reduce sampling variability and promote more consistent outputs across runs.

The empirical evaluation in this thesis is limited to this single LLM configuration. Consequently, observed agreement levels and failure modes are interpreted as results for the evaluated model and assessment setup rather than as model-independent properties of LLM-based FAIRness assessment. Cross-model evaluation is considered as an extension of the present study.

3.5.4 Prompt Design

The LLM is provided with the repository name and README content as input and instructed to score each dimension according to the defined rubric. The prompt specifies that the model must evaluate only evidence explicitly present in the README text, and must not assume content exists behind external links. Each score must be accompanied by a confidence level (HIGH, MEDIUM, or LOW) and a one-sentence reasoning statement citing specific README evidence.

The final prompt was developed through iterative experimentation on the pilot repository set, with refinements improving the specificity of dimension definitions, particularly for D5, where generic documentation links were being incorrectly treated as usage examples.

3.5.5 Implementation

The LLM-based scorer uses the Ollama HTTP API with structured JSON output enforced through a JSON schema supplied to the API, ensuring that every response conforms to the expected structure and that scores are restricted to the permitted values (0, 1, or 2). README content is supplied directly to the model as assessment evidence. To bound the amount of input passed to the model and keep preprocessing consistent across repositories, the implementation applies a fixed upper limit of 12,000 characters. For READMEs exceeding this threshold, only the first 12,000 characters are supplied to the model.

The threshold is a practical implementation-level preprocessing choice rather than a limit derived from the model's maximum context capacity or a threshold selected

through experimental optimisation. Character count was used because it provides a simple and deterministic way to bound input without introducing an additional tokenization step. However, language models process tokens rather than characters, and the relationship between character count and token count varies with the content of the README. A token-based limit would therefore provide a more direct measure of the actual model input size.

The implementation also applies the limit as a sequential cutoff rather than selecting content according to README structure. This approach is reproducible across repositories but may exclude relevant documentation when important sections occur after the cutoff. A section-aware strategy that identifies and preserves relevant sections, such as installation, usage, dependencies, examples, and contribution guidance, would be more closely aligned with the D1–D6 assessment dimensions.

Within the 30-repository ground-truth corpus, only one README (`Ouranosinc/xclim`) exceeded the 12,000-character threshold, with a total length of 14,262 characters. Inspection of the excluded portion showed that it consisted of reStructuredText definitions for images, badges, logos, and associated links rather than substantive documentation corresponding to D1–D6. No relevant assessment evidence was therefore identified as lost through truncation in this case. Nevertheless, this observation concerns a single repository and does not establish that fixed character-based truncation is generally safe for longer READMEs. Token-aware and section-aware input handling are therefore more principled alternatives for future versions of the framework. Confidence values and reasoning statements were retained to support qualitative error analysis but were not included in the quantitative scoring metrics.

3.5.6 External Link Policy

The LLM receives only the README text as input and cannot access external documentation linked from the README. Consequently, linked resources not reproduced in the README are treated as partial evidence (score 1) rather than complete evidence (score 2). The same README-only interpretation was applied during manual annotation to ensure a fair comparison between human and automated assessment. This design choice is documented as a limitation and is considered when interpreting differences between LLM and manual annotations.

3.5.7 Output

The LLM scorer produces a CSV file containing one row per repository with scores, confidence levels, and reasoning for each of the six dimensions, together with the model name and a timestamp. The scorer was applied to all 30 ground truth repositories, producing the LLM-based assessment results used in the evaluation presented in Chapter 5.

3.6 Repository FAIRness Profile

The hybrid assessment framework reports the outputs of the rule-based and LLM-based components together as a unified FAIRness profile for each repository. This representation maintains the interpretability of both structural FAIR indicators and qualitative documentation characteristics, allowing users to identify strengths and weaknesses across different aspects of software FAIRness.

The rule-based component contributes six binary indicator scores (F1–R2) covering structural metadata compliance, together with an aggregated rule-based total. The LLM-based component contributes six documentation quality scores (D1–D6) on a 0–2 ordinal scale, each accompanied by a confidence level and a reasoning statement.

Rather than aggregating these heterogeneous structural and semantic indicators into a single scalar score, the framework preserves them as separate dimensions. This avoids introducing arbitrary weights across dimensions with fundamentally different evidence types — binary file-presence checks versus qualitative, LLM-derived documentation judgments — and prevents this incomparable evidence from being collapsed into a potentially misleading overall FAIRness value. Both sets of scores are instead reported together within a single structured record per repository, stored as a row in a CSV output file.

This parallel representation provides more actionable feedback than a single aggregated score would allow. A repository may score highly on structural metadata indicators while exhibiting weak documentation quality, or conversely may have well-written documentation but lack persistent identifiers or structured citation metadata. Presenting both dimensions separately makes these patterns visible and directly supports the identification of specific FAIRness gaps.

The resulting hybrid FAIRness profiles constitute the primary output of the proposed framework and are used as the basis for the evaluation presented in Chapter 5.

3.7 Evaluation Strategy

The evaluation strategy assesses the accuracy, reliability, and added value of the proposed hybrid assessment framework through both quantitative and qualitative methods. The evaluation is structured around the two research questions defined in Chapter 1 and is conducted against the 30-repository ground-truth corpus described in Section 3.3.

3.7.1 Ground Truth Comparison

The primary evaluation compares the automated scores produced by both the rule-based and LLM-based components against the manual ground truth annotations. For the rule-based indicators (F1–R2), which are binary, performance against the manual ground truth is evaluated using precision, recall, and F1-score. Exact agree-

ment and Cohen's unweighted Kappa are additionally reported to characterise agreement between assessments. For the LLM-based dimensions (D1–D6), which use a 0–2 ordinal scale, agreement is measured using exact match accuracy, Mean Absolute Error (MAE), and Cohen's weighted Kappa per dimension, where partial agreement is given proportional credit.

3.7.2 Inter-Annotator Agreement

To assess the reliability of the manual ground truth annotations, inter-annotator agreement between the primary annotator and the second annotator was quantified using Cohen's unweighted Kappa for binary indicators and Cohen's weighted Kappa for ordinal dimensions. Exact agreement was also reported, particularly because Kappa is undefined when both annotators assign only a single class. These results establish the reliability of the manual reference annotations and contextualise the agreement observed between the automated assessments and the ground truth.

3.7.3 Baseline Comparison

The proposed approach is compared with existing FAIR assessment tools. A direct comparison with howfairis is performed on the ground truth repositories, examining indicator coverage, agreements and disagreements, and FAIRness aspects not represented by the existing tool. F-UJI is separately evaluated in a pilot experiment by submitting plain GitHub repository URLs directly, to investigate whether a dataset-oriented FAIR assessment tool can serve as a meaningful quantitative baseline when applied to research software repositories without a mediating dataset record.

3.7.4 Qualitative Error Analysis

In addition to quantitative agreement metrics, representative cases of agreement and disagreement between the automated assessments and the manual annotations are analysed. The confidence levels and reasoning statements produced by the LLM are used to help interpret discrepancies and identify common sources of scoring differences.

3.7.5 Confidence Calibration

The usefulness of the LLM's self-reported confidence labels is evaluated by comparing prediction correctness across HIGH, MEDIUM, and LOW confidence outputs. Agreement with the primary human annotations is examined both across all documentation dimensions and within individual dimensions where sufficient observations are available. This analysis assesses whether confidence can be interpreted as an indicator of prediction reliability or should instead be treated only as contextual information.

3.7.6 README Length and Verbosity Bias

To investigate whether LLM scoring errors are associated with README length, the difference between the LLM and human documentation scores is compared with the effective README length supplied to the model. README length is measured after applying the same 12,000-character input limit used during inference. Spearman's rank correlation is computed for the aggregate documentation score and separately for D1–D6. This analysis tests whether longer README files are systematically associated with higher LLM scores independently of the corresponding human assessments.

4 Implementation

4.1 System Architecture and Overview

The proposed framework is implemented as a modular pipeline of independent Python components organized around four main processing stages: repository data collection, rule-based assessment, LLM-based assessment, and hybrid profile generation. Each component exchanges data primarily through files stored on the local filesystem rather than through a shared database or continuously running application. This organization allows individual stages to be executed, inspected, and re-run independently. Where appropriate, batch-processing stages also include mechanisms for skipping already processed repositories, reducing unnecessary repeated work after interruption.

The implementation is written in Python. GitHub repository data is retrieved through the GitHub REST API using the `requests` library, while LLM inference is performed through a locally running Ollama instance. Standard file formats, primarily JSON and CSV, are used to persist intermediate and final results. Repository data is stored as per-repository snapshots containing the README content, general GitHub metadata, and an inventory of FAIR-related files. This file-based representation makes the evidence used by subsequent assessment stages directly inspectable and allows the scoring components to operate on previously collected snapshots without re-querying GitHub.

The overall data flow consists of four stages, illustrated in Figure 4.1

First, the collection component retrieves and prepares repository information from GitHub, producing the local repository snapshots used as input to subsequent stages. Second, the rule-based component evaluates structural evidence and produces six binary indicators (F1–R2). Third, the LLM-based component processes README documentation and produces six ordinal documentation scores (D1–D6), each accompanied by a confidence label and a short reasoning statement. Finally, the hybrid profile generation component joins the structural and documentation-based outputs by repository identifier and stores them as a unified repository profile while preserving the two assessment components as separate dimensions.

The rule-based assessment was executed for the complete 188-repository dataset. LLM assessment was executed for the 30-repository ground-truth corpus used for manual evaluation, and hybrid profiles were consequently generated for repositories for which both rule-based and LLM-based results were available. The implemented components therefore cover all stages required to construct the hybrid profile, while the experimental execution of the LLM-based component is restricted to the anno-

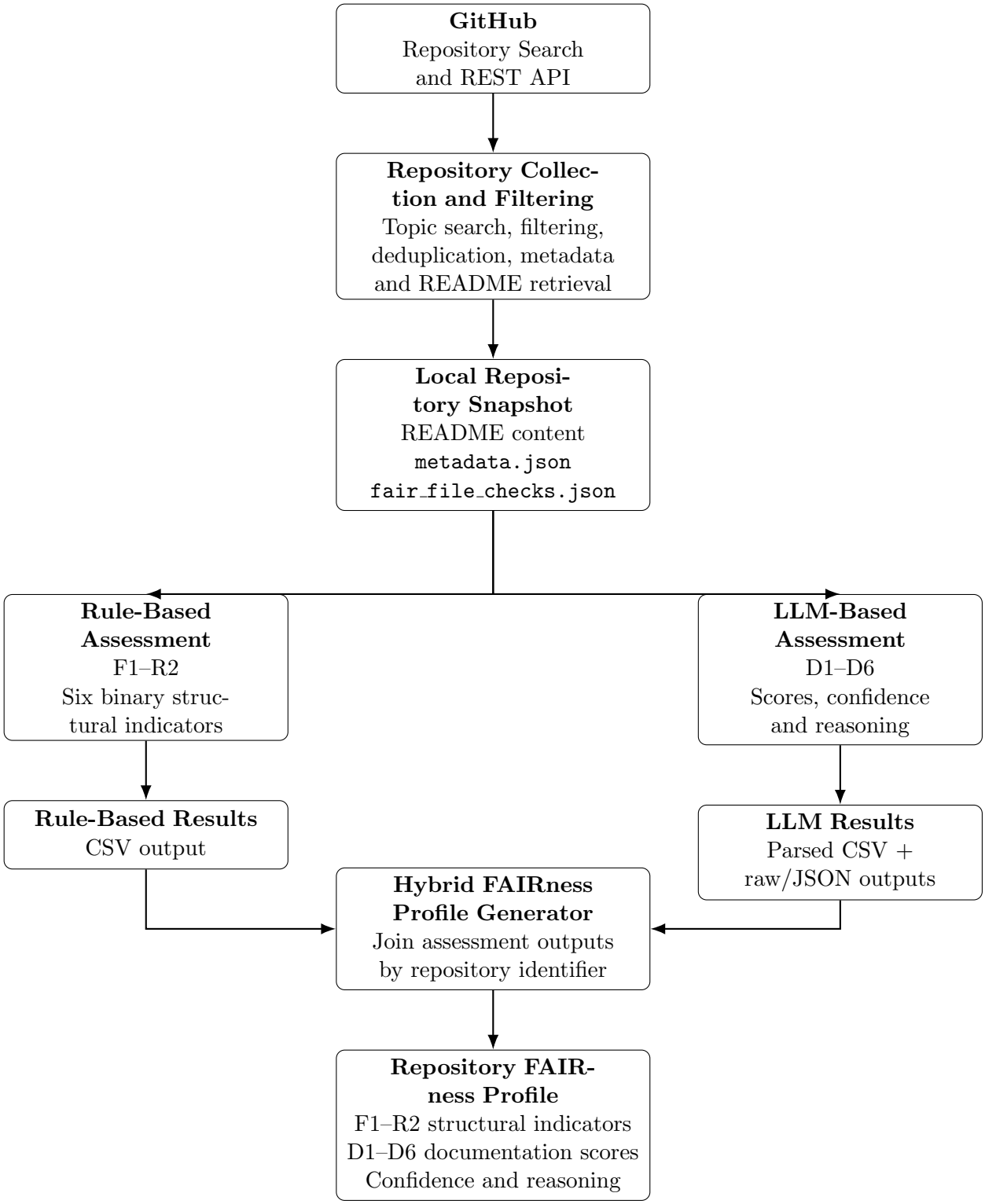


Figure 4.1: System architecture of the implemented hybrid FAIRness assessment pipeline.

tated evaluation corpus.

4.2 Repository Data Collection and Local Representation

Repository data collection is implemented as a two-phase process: repository discovery and selection, followed by per-repository data retrieval. Both phases interact with the GitHub REST API and are implemented as separate Python components. Repository discovery constructs the initial dataset, whereas the retrieval stage operates on the resulting repository list to obtain the artefacts required by the subsequent assessment components.

Repository discovery queries GitHub for repositories associated with the research domains defined during dataset construction. Candidate repositories are deduplicated by their GitHub repository identifier and filtered according to the inclusion and exclusion criteria described in Chapter 3. These criteria remove repositories that do not meet the intended characteristics of the research software dataset, including repositories identified as forks, archived projects, and repositories matching predefined exclusion conditions. The resulting repository list is persisted in machine-readable form and subsequently processed by the data retrieval stage.

For each selected repository, the retrieval component obtains three main types of information. First, README content is retrieved through GitHub's dedicated README endpoint and stored locally for subsequent documentation assessment. Second, the repository metadata returned by the GitHub repository endpoint is stored in a `metadata.json` file. This record contains the GitHub metadata required by later processing stages, including repository and licensing information. Third, the contents of the repository root are queried and compared against a predefined set of FAIR-related filenames. The resulting presence inventory is stored as `fair_file_checks.json`. Candidate files include README and license variants, citation and structured metadata files, dependency specifications, and contribution files.

The collected artefacts are organized as local repository snapshots. Each repository is assigned a directory based on its owner and repository name, containing the retrieved README, repository metadata, and FAIR-related file inventory. This representation decouples data collection from assessment: once a snapshot has been created, the rule-based and LLM-based components can operate on the locally stored evidence without repeatedly querying GitHub. It also makes the exact repository evidence used during an assessment directly inspectable.

The batch retrieval implementation includes basic support for resuming interrupted collection runs. Existing repository outputs are checked before retrieval, allowing repositories that have already been collected to be skipped and avoiding unnecessary repeated API requests. Individual artefacts are also checked before retrieval in the relevant collection routines. The resulting snapshots form the common

input representation used by the subsequent assessment components.

4.3 Rule-Based Assessment Implementation

The rule-based assessment component evaluates each repository snapshot against the six binary FAIR indicators (F1–R2) defined in Chapter 3, operating exclusively on the locally stored README, metadata, and file-inventory artefacts produced by the collection stage; no GitHub API access occurs during scoring.

Each indicator is implemented as an independent boolean check, drawing on one or more of three evidence sources: the precomputed file-presence inventory, the stored GitHub metadata record, and the README text itself. F2 (Structured Metadata) is evaluated by checking for recognized metadata filenames using both the stored file-presence inventory and the local repository snapshot. R1 (README Present) directly checks the snapshot for one of the recognized README filename variants. I1 (Dependency Specification) similarly evaluates recognized dependency and build files across multiple programming-language ecosystems. F1 (Persistent Identifier) searches the README together with any locally stored `CITATION.cff` and `codemeta.json` content for patterns associated with common persistent identifier formats, including doi.org URLs, Zenodo record and badge links, and Software Heritage identifiers. A1 (License) reflects the three evidence sources described in Chapter 3: a recognized license filename, a license field in the stored GitHub metadata, or explicit licensing language in the README text; the three sources are checked in this order, so that structured evidence takes precedence and README text is only relied upon when no file or metadata evidence is found. R2 (Citation Information) is satisfied either by the presence of a dedicated citation file or by matching one of several phrasings commonly used to request citation of a software project within the README text.

Where file-presence evidence is required, the implementation does not rely solely on the precomputed inventory. The corresponding path in the local repository snapshot is also checked, allowing a file-based indicator to be satisfied when the artefact exists locally but is absent from the stored presence record.

The scorer processes repositories in batch, reading each repository's snapshot folder in turn and producing a single six-indicator vector per repository, together with an aggregated rule-based total ranging from 0 to 6. Repositories whose snapshot is incomplete — for example, missing the metadata or file-inventory record — are skipped and reported rather than scored with incomplete evidence. Results are written to a single CSV file, with one row per repository, in which each of the six indicators and the aggregated total form separate columns. This output format is shared by both the full 188-repository dataset run and the 30-repository ground truth validation run.

4.4 LLM-Based Assessment Implementation

The LLM-based assessment component evaluates README documentation against the six documentation quality dimensions (D1–D6) defined in Chapter 3, using a locally hosted instance of Llama 3.1 8B served through Ollama.

For each repository, the README is located by checking a fixed set of recognized filenames in priority order, so that non-Markdown README variants are still picked up if present. The retrieved text is truncated to the first 12,000 characters where it exceeds this length, consistent with the preprocessing limit described in Chapter 3, before being inserted into a fixed prompt template together with the repository name. The prompt instructs the model to score each of the six dimensions according to the rubric criteria, to base each score only on evidence explicitly present in the README text, and to provide a confidence label (HIGH, MEDIUM, or LOW) and a one-sentence justification for every score based on the README evidence.

The scorer submits each request to Ollama's HTTP API rather than invoking the model through a command-line interface, with the sampling temperature set to zero and a structured JSON schema supplied as part of the request. The schema constrains the response to the six required dimensions, restricts each score to the integer values 0, 1, or 2, and restricts each confidence label to one of the three permitted values. On receiving a response, the implementation performs an additional validation pass independent of the schema — confirming that all six dimensions are present and that every score and confidence value falls within the expected range — before accepting the result. If a response is missing, malformed, or fails this validation, the request is retried once; if the retry also fails, the repository is recorded as failed and scoring proceeds to the next repository rather than halting the batch run.

Every raw model response is saved to disk separately for each attempt, regardless of whether it is subsequently parsed and validated successfully. For successfully scored repositories, the parsed and validated JSON representation is stored separately. This preserves the original model output for later inspection, including responses associated with unsuccessful attempts.

Batch execution is resumable at the level of individual repositories. Before submitting a repository for assessment, the implementation checks whether a previously parsed result is already available and skips repositories that have already been successfully processed. This allows interrupted runs to continue without repeating completed LLM inference.

Successfully scored repositories are appended incrementally to a single CSV output file as they complete, with one row per repository containing the score, confidence level, and reasoning text for each of the six dimensions, together with the model name and a timestamp. This output format matches the one described in Chapter 3 and is applied to all 30 ground truth repositories.

4.5 Hybrid Profile Generation

The hybrid profile generation component combines the outputs of the rule-based and LLM-based components into a single unified record per repository, without computing any combined or weighted score.

The component reads the rule-based and LLM-based result CSV files produced by the two preceding stages and indexes each by repository identifier. An inner join is then performed between the two sets of results, so that a hybrid profile is produced only for repositories present in both inputs. Because the LLM-based component was applied only to the 30-repository ground truth corpus, this join consequently restricts the set of generated hybrid profiles to those repositories, even though rule-based results exist for the complete 188-repository dataset.

For each matched repository, the resulting record concatenates the six rule-based indicator scores (F1–R2) and the aggregated rule-based total with the six LLM-based dimension scores (D1–D6), each accompanied by its confidence level and reasoning text, the aggregated LLM total, and the model name and timestamp associated with the LLM assessment. No arithmetic combination or weighting is applied across the two sets of scores at any point in this process; the structural and documentation-based dimensions are retained as distinct fields within the same record, consistent with the design rationale described in Chapter 3. A single timestamp is recorded for the batch as a whole, and repositories are written in a fixed, sorted order to ensure that the output is deterministic regardless of the order in which the two input files were produced.

The combined records are written to a single CSV file, with one row per repository, representing the hybrid FAIRness profile produced by the proposed framework. The profile provides a unified representation of the structural and documentation-based assessment results while preserving their distinct scoring schemes.

4.6 Example Repository Profile

To illustrate the output produced by the framework, this section presents the complete hybrid FAIRness profile generated for `astropy/astropy`, one of the 30 ground truth repositories. This repository was selected because its profile shows a clear contrast between the two assessment components: it achieves the maximum possible rule-based score while receiving a comparatively low LLM-based documentation score, illustrating the kind of gap that a purely structural assessment would not reveal.

4.6.1 Rule-Based Indicators

Table 4.1 shows the six rule-based indicator scores produced for this repository.

The repository satisfies all six structural indicators, reflecting the presence of a persistent identifier, a structured metadata file, a recognized license, a dependency

Table 4.1: Rule-based indicator scores for `astropy/astropy`.

Indicator	Score
F1 — Persistent Identifier	1
F2 — Structured Metadata	1
A1 — License	1
I1 — Dependency Specification	1
R1 — README Present	1
R2 — Citation Information	1
Rule-based total	**6 / 6**

specification file, a README, and explicit citation information.

4.6.2 LLM-Based Documentation Dimensions

Table 4.2 shows the six LLM-based documentation quality scores, together with the confidence level and reasoning text produced for each dimension.

4.6.3 Interpretation

The two components of the profile convey different information about the same repository. The rule-based component indicates that the repository satisfies all six structural indicators evaluated by the framework. The LLM-based component, in contrast, indicates that a new user attempting to reuse the software from the README alone would find limited concrete guidance — installation is reduced to a single command with no stated prerequisites, no inline usage example is provided, and required dependencies are not listed. The purpose description receives partial credit, while the contribution guidance (D6) receives the maximum score.

The example demonstrates how the hybrid profile preserves information that would be obscured by considering either assessment component in isolation. Although `astropy/astropy` satisfies all six rule-based indicators, its documentation-quality assessment is substantially lower. Retaining the structural and documentation-based scores as separate dimensions therefore makes this distinction explicit rather than reducing the assessment to a single aggregated FAIRness score.

4.7 Implementation Summary

This chapter has described the implementation of the proposed hybrid FAIRness assessment framework: a modular pipeline consisting of repository data collection, a rule-based assessment component, an LLM-based assessment component, and a hybrid profile generator that combines their outputs without computing a combined score. Repository data is persisted as local snapshots, while the assessment

Table 4.2: LLM-based documentation scores for `astropy/astropy`.

Dimension	Score (Conf.)	Reasoning
D1 — Installation	0 (HIGH)	The README provides only a single line of installation information (`pip install astropy`), without mentioning required prerequisites.
D2 — Usage	0 (HIGH)	There is no concrete command, code snippet, workflow, or executable example in the README to demonstrate how to use the software.
D3 — Purpose	1 (MEDIUM)	The README explains that Astropy is a community effort to develop a single core package for astronomy in Python, but it does not clearly explain its intended domain or audience.
D4 — Dependencies	0 (HIGH)	There is no information about required and/or optional dependencies provided in human-readable form in the README.
D5 — Examples	1 (MEDIUM)	The README mentions GitHub Codespaces as a way to start developing Astropy, but it does not contain a meaningful usage example, nor explicitly link to an example gallery or tutorial demonstrating software use.
D6 — Contribution	2 (HIGH)	The README provides contribution guidance and at least one support or community channel (Slack), as well as multiple ways to contribute, including code submission, feedback, and documentation updates.
LLM total		**4 / 12**

components produce structured intermediate and final outputs. This file-based organization allows the individual processing stages to be executed, inspected, and re-run independently.

The rule-based component was applied to the complete 188-repository dataset, while the LLM-based component and the resulting hybrid profiles were produced for the 30-repository ground truth corpus. The example profile presented in Section 4.6 illustrates the kind of output the framework produces for an individual repository and the distinct information conveyed by its two assessment components.

The following chapter evaluates the accuracy and reliability of both components against the manually annotated ground truth and compares the proposed approach with existing FAIR assessment tools.

5 Evaluation and Results

5.1 Experimental Setup

This chapter presents the quantitative evaluation of the proposed hybrid FAIRness assessment framework. The results assess the reliability of the framework and compare its outputs against manually annotated ground truth and an existing baseline approach.

5.1.1 Dataset

The full dataset comprises 188 research software repositories collected from GitHub across 10 research domains, as described in Chapter 3. In addition, a manually annotated corpus of 30 repositories was constructed to provide ground truth for quantitative evaluation. The annotated corpus was selected to provide diversity across research domains and programming languages, including repositories implemented in Python, C++, Java, MATLAB, Fortran, Julia, Go, Rust, and Groovy.

Ten of the 30 annotated repositories were initially used during a pilot phase to refine the annotation guidelines and assessment procedure. These repositories were retained in the final annotated corpus. To assess the effect of this original pilot overlap, the remaining 20 repositories were additionally analysed as a pilot-exclusion subset. For the LLM-based dimensions, these repositories had not been used during pilot-phase refinement and therefore provide an independent sensitivity check of LLM–human agreement. For the rule-based indicators, the same subset is used only as an internal consistency check because some rule-based annotations were corrected during final scorer validation.

5.1.2 Evaluation Metrics

The following metrics are used throughout this chapter:

Exact Match Accuracy The proportion of repositories for which the automated score exactly matches the manual annotation. Reported as a percentage.

Mean Absolute Error (MAE) The average absolute difference between the automated assessment and the corresponding manual annotation, computed per dimension across all 30 ground truth repositories. Applied to the ordinal LLM-based dimensions (D1–D6).

Cohen's Unweighted Kappa Used for binary rule-based indicators (F1–R2), where any disagreement is treated equally regardless of magnitude.

Cohen's Linearly Weighted Kappa Used for ordinal AI-based dimensions (D1–D6), where partial agreement is given proportional credit. A disagreement of 1 point is penalised less than a disagreement of 2 points.

Kappa values are interpreted following the scale proposed by Landis and Koch [7]: values below 0.20 are considered slight, 0.21–0.40 fair, 0.41–0.60 moderate, 0.61–0.80 substantial, and above 0.80 almost perfect agreement.

5.1.3 Models and Tools

The LLM-based component uses the `llama3.1:8b` model executed locally via the Ollama HTTP API at temperature 0. The model was selected because it can be executed locally without external API calls, ensuring reproducibility and avoiding data transfer outside the experimental environment. Structured JSON output is enforced through a schema to constrain scores to the permitted values (0, 1, or 2). The rule-based component operates deterministically on locally stored repository snapshots.

The howfairis tool is used as a software-oriented baseline comparison. F-UJI, an automated FAIR assessment tool designed primarily for research data objects, is additionally used in a pilot experiment to investigate the applicability of an external, data-oriented FAIR assessment baseline to research software repositories.

5.1.4 Experimental Environment

The evaluation pipeline was implemented in Python and executed locally on Windows using Ollama for LLM inference. The LLM inference was performed using the `llama3.1:8b` model (4.9 GB). All scripts, data, and results are stored in the project repository to support reproducibility.

5.1.5 Evaluation Procedure

The evaluation was conducted in the following order:

1. The thesis author manually annotated 30 ground truth repositories against the finalised annotation guidelines (v1.4).

2. A second annotator independently annotated the same 30 repositories using the same guidelines.

3. Inter-annotator agreement was computed between the two human annotators to validate the reliability of the ground truth.

4. The LLM scorer was applied to the same 30 repositories and the outputs were compared against both annotators.

5. The rule-based scorer was applied to all 188 repositories to produce a complete set of structural FAIR indicator scores.

6. Hybrid FAIRness profiles were generated by combining rule-based and LLM scores for the 30 ground truth repositories.

7. The howfairis baseline was executed on the 30 ground truth repositories and compared against the proposed framework.

8. F-UJI was applied directly to a subset of five ground truth repositories to investigate its applicability as an external data-oriented FAIRness assessment baseline for research software.

9. As a sensitivity analysis, final rule-based and LLM-based performance was recomputed on the 20-repository pilot-exclusion subset after excluding the 10 repositories used during pilot-phase refinement.

5.2 Ground Truth Validation

5.2.1 Annotation Process

Manual annotations were created by the thesis author for all 30 ground truth repositories following the finalised annotation guidelines (v1.4), as described in Chapter 3. Each repository was annotated by examining the repository root file structure and README content. Rule-based indicators (F1–R2) were assigned binary scores according to the annotation guidelines based on evidence obtained from the repository structure and README content. AI-based dimensions (D1–D6) were scored on a 0–2 ordinal scale based on the clarity and completeness of the README documentation.

To assess annotation reliability, a second annotator independently annotated the same 30 repositories using identical guidelines and pre-fetched repository data, without access to the primary annotations.

5.2.2 Inter-Annotator Agreement

Inter-annotator agreement between the two annotators was quantified using Cohen's unweighted Kappa for binary rule-based indicators and Cohen's linearly weighted Kappa for ordinal AI-based dimensions. Table 5.1 presents the results.

Overall, the agreement results indicate that both the binary rule-based indicators and the ordinal documentation quality dimensions can be annotated consistently using the proposed guidelines.

Table 5.1: Inter-annotator agreement for rule-based indicators and AI-based dimensions (n=30).

Indicator/Dimension	Type	κ	Exact	Interpretation
F1 — Persistent identifier	Binary	0.710	90.0%	Substantial
F2 — Structured metadata	Binary	1.000	100.0%	Almost perfect
A1 — License	Binary	undef.	100.0%	Single class
I1 — Dependency file	Binary	0.634	93.3%	Substantial
R1 — README present	Binary	undef.	100.0%	Single class
R2 — Citation information	Binary	0.658	83.3%	Substantial
Rule macro-avg		**0.750**		
D1 — Installation	Ordinal	0.539	70.0%	Moderate
D2 — Usage	Ordinal	0.747	83.3%	Substantial
D3 — Purpose	Ordinal	0.789	96.7%	Substantial
D4 — Dependencies	Ordinal	0.743	80.0%	Substantial
D5 — Examples	Ordinal	0.585	70.0%	Moderate
D6 — Contribution	Ordinal	0.849	90.0%	Almost perfect
AI macro-avg		**0.709**		

The AI-based dimensions achieved a macro-average weighted Kappa of 0.709, indicating substantial overall agreement between annotators. The strongest agreement was observed for D6 (contribution information, κ=0.849) and D3 (purpose description, κ=0.789). The weakest agreement was observed for D1 (installation clarity, κ=0.539) and D5 (examples and tutorials, κ=0.585), suggesting greater ambiguity when assessing partially documented functionality or documentation provided through external links.

For the rule-based indicators, F2 achieved perfect agreement, while F1, I1, and R2 showed substantial agreement. Two indicators are mathematically undefined: R1 (README present), because all 30 repositories have a README file, and A1 (license), because both annotators agreed on the presence of a license for all 30 repositories following a correction described below. Both indicators demonstrate consistent annotation in practice, as reflected by their 100% exact agreement.

The rule-based macro-average of 0.750 excludes the two undefined values (A1 and R1).

I1 (dependency file present) initially showed disagreement on seven repositories, predominantly projects using non-Python language ecosystems (C++, Java, Go, Rust, Groovy, Julia), following the pattern that the primary annotator assigned a score of 0 while the second annotator assigned a score of 1. Investigation traced this to an incomplete file-detection schema used during ground-truth collection: the collector script for the 30 ground-truth repositories predated the expanded dependency-file list (including `go.mod`, `Cargo.toml`, `pom.xml`, `build.gradle`, and `Project.toml`) later adopted for the full 188-repository collection, and consequently could not detect language-specific dependency files for non-Python projects. Five

of the seven affected repositories were confirmed via the corrected file inventory to genuinely contain a dependency specification file, and the corresponding primary annotations were corrected accordingly. Following this correction, I1 agreement improved from κ=0.286 (fair) to κ=0.634 (substantial).

A1 (license) was also corrected during validation: one repository (`rust-ml/linfa`) had been annotated as lacking a license, despite both the GitHub repository metadata and the README text explicitly identifying an Apache-2.0 license. This annotation was corrected, resulting in full agreement between annotators for A1.

These inter-annotator agreement results demonstrate sufficient reliability of the manual annotations to serve as the ground truth for the subsequent evaluation.

5.3 Rule-Based Assessment Results

The rule-based scorer was applied to all 188 repositories in the full dataset, producing binary scores for each of the six FAIR indicators (F1–R2). This section presents the distribution of rule-based assessment results across the full dataset and evaluates the rule-based component against the manually annotated ground truth repositories.

5.3.1 Full Dataset Results

Table 5.2 summarises the frequency of positive scores for each indicator across all 188 repositories.

Table 5.2: Rule-based indicator frequencies across the full dataset (n=188).

Indicator	Description	Positive	Percentage
F1	Persistent identifier	95	51%
F2	Software metadata	58	31%
A1	License	182	97%
I1	Dependency file	123	65%
R1	README present	188	100%
R2	Citation information	79	42%

All 188 repositories contain a README file (R1=100%), and 97% have a license file (A1=97%). In contrast, dedicated software metadata files such as `CITATION.cff` or `codemeta.json` are present in only 31% of repositories (F2), and persistent identifiers such as DOIs or Zenodo badges are present in only 51% (F1). Citation information is available in 42% of repositories (R2), suggesting that fewer than half of the evaluated repositories provide explicit guidance on how to cite the software. Overall, the results indicate that basic repository documentation practices, such as providing README and license files, are widespread, whereas richer FAIR-related metadata and citation support remain considerably less common.

The mean rule-based score across the full dataset was 3.86 out of a maximum possible score of 6. The score distribution is shown in Table 5.3.

Table 5.3: Distribution of rule-based total scores across the full dataset (n=188).

Score	Count	Percentage
6/6	15	8%
5/6	43	23%
4/6	61	32%
3/6	42	22%
2/6	23	12%
1/6	4	2%

The score distribution spans the full range from 1 to 6, indicating substantial variability in structural FAIR compliance across the dataset. Only 15 repositories (8%) achieve full compliance across all six rule-based indicators, while 27 repositories (14%) score 2 or below, indicating significant metadata gaps.

5.3.2 Validation Against Ground Truth

The rule-based scorer was validated against the final manual annotations for the 30 ground truth repositories. During validation, differences between automated predictions and manual annotations were inspected individually and resolved according to the finalised annotation guidelines (v1.4).

This process identified both implementation-level detection gaps and manual annotation inconsistencies, particularly affecting the citation (R2), persistent identifier (F1), and license (A1) indicators. The rule-based detection logic was refined to recognise additional valid citation and persistent identifier patterns, including `CITATION.cff` files, generic DOI representations, and explicit citation wording. Manual annotations that were inconsistent with the final rubric were corrected and documented in the corresponding repository annotation files.

After these refinements, the final rule-based scorer achieved exact agreement with the final ground truth annotations across all 30 repositories. Precision, recall, and F1-score were 1.000 for each of the six binary indicators (F1–R2), as shown in Table 5.4.

Table 5.4: Rule-based validation against manual ground truth (n=30).

Indicator	Precision	Recall	F1-score
F1 — Persistent identifier	1.000	1.000	1.000
F2 — Software metadata	1.000	1.000	1.000
A1 — License	1.000	1.000	1.000
I1 — Dependency file	1.000	1.000	1.000
R1 — README present	1.000	1.000	1.000
R2 — Citation information	1.000	1.000	1.000

The final results demonstrate that the rule-based implementation faithfully repro-

duces the operational definitions encoded in the finalised manual annotation rubric for the evaluated ground-truth corpus. Because scorer refinements and corrections to some rule-based annotations were made during final validation, the resulting perfect agreement should not be interpreted as an unbiased estimate of performance on unseen repositories. The additional pilot-exclusion analysis in Section 5.4.3 therefore serves as an internal consistency check for the rule-based component rather than as independent validation.

5.4 LLM-Based Assessment Results

The LLM-based scorer was applied to all 30 ground truth repositories using the `llama3.1:8b` model at temperature 0. This section presents the agreement between LLM-generated documentation quality scores and manual annotations from both annotators, and identifies systematic patterns in LLM scoring behaviour.

5.4.1 LLM vs Primary Annotator

Table 5.5 presents the agreement between LLM scores and the primary annotator across all six documentation quality dimensions.

Table 5.5: LLM vs primary annotator agreement (n=30).

Dimension	κ	**Exact**	**MAE**	**Interpretation**
D1 — Installation	0.605	70.0%	0.333	Moderate
D2 — Usage	0.274	46.7%	0.567	Fair
D3 — Purpose	−0.071	86.7%	0.133	Prevalence effect
D4 — Dependencies	0.480	60.0%	0.433	Moderate
D5 — Examples	0.192	36.7%	0.700	Slight
D6 — Contribution	0.494	63.3%	0.400	Moderate
Macro-avg	**0.329**			Fair

5.4.2 LLM vs Second Annotator

Table 5.6 presents the agreement between LLM scores and the second annotator.

5.4.3 Pilot-Exclusion Sensitivity Analysis

Because 10 of the 30 annotated repositories had been used during pilot-phase refinement, final-system results were additionally examined after excluding these repositories. The resulting 20-repository subset provides an independent sensitivity check for the LLM-based dimensions, which were not modified during this validation process. For the rule-based component, however, the analysis should be interpreted more cautiously: several manual rule-based annotations within this subset were corrected

Table 5.6: LLM vs second annotator agreement (n=30).

Dimension	κ	**Exact**	**MAE**	**Interpretation**
D1 — Installation	0.480	70.0%	0.433	Moderate
D2 — Usage	0.276	50.0%	0.600	Fair
D3 — Purpose	−0.056	86.7%	0.167	Prevalence effect
D4 — Dependencies	0.338	53.3%	0.567	Fair
D5 — Examples	0.156	30.0%	0.867	Slight
D6 — Contribution	0.454	60.0%	0.433	Moderate
Macro-avg	**0.275**			Fair

during final scorer validation after inconsistencies in the annotation evidence and detection schema were identified. The rule-based 20-repository result is therefore reported as an internal consistency check rather than as independent validation.

For the rule-based component, precision, recall, and F1-score were 1.000 for all six indicators after exclusion of the 10 original pilot repositories. Because some rule-based annotations in this subset were corrected during final validation, this result confirms consistency between the final scorer and finalised annotation rubric but should not be interpreted as an unbiased estimate of performance on unseen repositories.

For the LLM-based component, Table 5.7 compares agreement with the primary annotator on the full annotated corpus and the pilot-exclusion subset.

Table 5.7: LLM agreement with the primary annotator on the full annotated corpus and the 20-repository pilot-exclusion subset.

Dimension	κ **(n=30)**	κ **(pilot-excl., n=20)**
D1 — Installation	0.605	0.659
D2 — Usage	0.274	0.474
D3 — Purpose	−0.071	−0.071
D4 — Dependencies	0.480	0.337
D5 — Examples	0.192	0.286
D6 — Contribution	0.494	0.556
Macro-avg	**0.329**	**0.373**

For the LLM-based component, excluding the 10 pilot repositories did not reduce agreement with the primary annotator. The macro-average weighted Kappa increased slightly from 0.329 on the complete 30-repository corpus to 0.373 on the 20-repository pilot-exclusion subset. Since neither the LLM scorer nor the D1–D6 annotations in this subset were modified as part of the rule-based validation corrections, this comparison provides an independent sensitivity check for the effect of the original pilot overlap on LLM–human agreement. Agreement increased for D1, D2, D5, and D6, remained unchanged for D3, and decreased for D4.

Overall, excluding the repositories involved in the original pilot did not weaken

the LLM-based findings. The rule-based results likewise remain internally consistent after pilot exclusion, although their perfect performance should not be interpreted as independent generalisation evidence because rule-based annotation corrections were made during final validation.

5.4.4 Systematic Scoring Patterns

Analysis of the direction of disagreements between LLM and human scores reveals a consistent pattern: the LLM assigned lower scores than human annotators more frequently than higher scores across almost all dimensions and both annotators. Table 5.8 summarises the direction of disagreements for the primary annotator comparison.

Table 5.8: Direction of LLM scoring disagreements relative to primary annotator (n=30). LLM lower = LLM score < human score; LLM higher = LLM score > human score.

Dimension	LLM lower	LLM higher
D1 — Installation	8	1
D2 — Usage	14	2
D3 — Purpose	2	2
D4 — Dependencies	8	4
D5 — Examples	17	2
D6 — Contribution	9	2

The most pronounced underscoring occurs in D5 (examples and tutorials, 17 cases lower vs 2 higher) and D2 (usage clarity, 14 cases lower vs 2 higher). These dimensions are most affected by the external link policy adopted in this study: the LLM evaluates only README text and cannot access external documentation linked from it. Repositories that reference tutorials, usage guides, or example galleries via external links receive lower LLM scores than human annotators who take the existence of those resources into account.

D3 (purpose description) shows a negative Kappa (-0.071) despite 86.7% exact agreement, reflecting a prevalence effect: 28 of 30 repositories received a human score of 2 for purpose description, leaving insufficient variation for meaningful Kappa computation.

D1 (installation clarity) achieved the strongest agreement with the primary annotator (κ=0.605), consistent with the observation that installation instructions are typically provided inline in README files and are therefore fully accessible to the LLM.

5.4.5 Comparison with Inter-Annotator Agreement

Table 5.9 compares LLM-human agreement with human-human agreement.

Table 5.9: Agreement summary: human-human vs LLM-human (macro-avg weighted κ, AI dimensions only).

Comparison	Macro-avg κ
Human A1 vs Human A2	0.709
LLM vs Human A1	0.329
LLM vs Human A2	0.275

Human-human agreement (κ=0.709) substantially exceeds LLM-human agreement (κ=0.329 and 0.275), confirming that the LLM assessment is less reliable than human annotation for this task. The consistency of the pattern across both annotator comparisons suggests that the findings reflect genuine LLM behaviour rather than idiosyncrasies of one annotator's scoring style.

5.4.6 Confidence Calibration

The LLM scorer additionally reports a self-assessed confidence level (HIGH, MEDIUM, or LOW) for each documentation quality score. To evaluate whether these confidence labels correspond to actual agreement with the human ground truth, the 180 individual dimension-level predictions (30 repositories $\times$ 6 dimensions) were grouped by reported confidence level. Exact agreement, mean absolute error (MAE), and linearly weighted Cohen's Kappa were then computed within each confidence group.

Table 5.10 summarises the results across all six documentation dimensions.

Table 5.10: LLM confidence calibration against the primary human annotations across all documentation dimensions.

Confidence	n	Exact	MAE	κ
HIGH	113	56.6%	0.460	0.520
MEDIUM	60	71.7%	0.283	0.478
LOW	7	28.6%	1.143	−0.120

The reported confidence levels were not monotonically associated with prediction accuracy. Predictions labelled MEDIUM achieved the highest exact agreement (71.7%) and the lowest MAE (0.283), whereas HIGH-confidence predictions achieved lower exact agreement (56.6%) and a higher MAE (0.460). LOW-confidence predictions performed worst, with 28.6% exact agreement and an MAE of 1.143, although this group contained only seven predictions and should therefore be interpreted cautiously.

The dimension-level results further show that confidence reliability varies across assessment dimensions. For example, HIGH-confidence predictions for D3 (purpose) achieved 94.4% exact agreement, whereas HIGH-confidence predictions for D5 (examples) achieved only 29.4%. Similarly, MEDIUM-confidence predictions outperformed HIGH-confidence predictions in exact agreement for D1, D2, D4, and D5.

This pattern is consistent with the systematic scoring behaviour identified in Section 5.4.4, particularly for D2 and D5. In these dimensions, the model frequently assigned lower scores when relevant usage guidance or examples were provided through external documentation rather than directly in the README. The poor accuracy of HIGH-confidence predictions in these dimensions therefore suggests that the model can be confident even when its assessment differs from the human interpretation of the rubric.

These results indicate that the model's self-reported confidence provides some useful information at the low-confidence extreme but is not a consistently calibrated indicator of prediction correctness. In particular, a HIGH confidence label should not be interpreted as guaranteeing stronger agreement with human assessment. Confidence should therefore be retained as contextual information in the hybrid FAIRness profile rather than treated as a quantitative reliability estimate.

5.4.7 README Length and Verbosity Bias

To investigate whether the LLM systematically assigned higher scores to repositories with longer README files, README length was compared with the difference between the LLM and human assessment scores. For each repository, the score difference was calculated as the LLM total minus the corresponding human total. For the correlation analysis, README length was measured as the number of words in the effective README text supplied to the LLM, after applying the same 12,000-character preprocessing limit used during inference.

Spearman's rank correlation showed a weak negative and non-significant relationship between README length and scoring difference ($\rho = -0.169$, $p = 0.371$, $n = 30$). Thus, longer README files were not associated with systematically inflated LLM scores in the evaluated sample.

The same analysis was performed separately for each documentation dimension. No statistically significant correlation was observed for D1–D6, with coefficients ranging from $\rho = -0.180$ for dependency documentation (D4) to $\rho = 0.060$ for installation documentation (D1). Notably, D2 (usage) and D5 (examples) — the dimensions identified in Section 5.4.4 as most affected by systematic underscoring due to the external link policy — showed weak negative correlations ($\rho = -0.110$ and $\rho = -0.135$, respectively), the opposite direction from what a verbosity bias would predict. These results provide no evidence that README length offers a competing explanation for the underscoring observed in those dimensions or of a systematic verbosity bias within the evaluated ground-truth set.

Only one of the 30 README files exceeded the 12,000-character input limit used by the LLM assessment pipeline. Inspection of the excluded portion showed that it consisted of reStructuredText definitions for images, badges, logos, and associated links rather than substantive documentation corresponding to D1–D6. No relevant assessment evidence was therefore identified as lost through truncation in this case. Nevertheless, this observation concerns only one repository, and the fixed sequential cutoff may remove relevant sections in other long READMEs. The absence of a

significant relationship in the present ground-truth sample should therefore not be interpreted as evidence that verbosity or truncation effects cannot occur in a broader repository population.

5.5 Hybrid FAIRness Profile Analysis

The hybrid FAIRness profile presents the rule-based indicators (F1–R2) and LLM-based documentation quality scores (D1–D6) together in a single structured record per repository while preserving them as separate assessment dimensions. No overall scalar FAIRness score is computed across the two components. This section analyses the profiles for the 30 ground truth repositories and examines the relationship between the two assessment dimensions.

5.5.1 Score Distributions

Table 5.11 presents the mean scores for both components across the 30 ground truth repositories.

Table 5.11: Mean scores for rule-based and LLM-based components across the 30 ground truth repositories.

Component	Mean Score	Maximum
Rule-based total	4.30	6
LLM total	6.70	12

The mean rule-based score across the 30 ground truth repositories was 4.30 out of 6, slightly higher than the mean across the full 188-repository dataset (3.86). The mean LLM total was 6.70 out of 12.

These component-level totals are reported only as descriptive summaries within each assessment component. They are not combined or interpreted as a single overall FAIRness score.

5.5.2 Relationship Between Rule-Based and LLM Scores

To examine whether repositories that score highly on structural FAIR indicators also score highly on documentation quality, Spearman's rank correlation was computed between the rule-based total and the LLM total across the 30 ground truth repositories. The result ($\rho = 0.175$, $p = 0.356$) indicates a weak positive association that was not statistically significant.

No clear monotonic association was therefore detected between the two components in the evaluated sample. This weak relationship supports treating structural FAIR compliance and documentation quality as complementary assessment dimensions. A repository may achieve a high rule-based score through the presence of formal metadata files while providing limited inline documentation, or conversely may

offer well-written README content while lacking persistent identifiers or structured citation metadata. The hybrid profile captures both dimensions simultaneously, providing a more complete picture than either component alone.

5.5.3 Complementary Assessment Dimensions

The evaluated repositories exhibited complementary assessment patterns. Repositories with high rule-based scores but comparatively low LLM scores possess strong structural FAIR metadata while providing limited inline documentation. Conversely, other repositories achieve higher documentation scores despite lacking formal metadata such as `CITATION.cff` files or persistent identifiers.

By presenting both dimensions in parallel, the hybrid profile allows users to identify specific FAIRness gaps and prioritise improvements accordingly, rather than receiving a single aggregated score that obscures the nature of any deficiencies.

5.5.4 Profile Interpretation

The hybrid FAIRness profile provides a more informative basis for repository assessment than either component alone. A repository with high scores in both components demonstrates both structural FAIR compliance and high-quality documentation. Conversely, a repository with a high rule-based score but a low LLM score may be formally compliant while remaining difficult to reuse in practice. Similarly, a repository with lower rule-based scores but higher LLM scores may provide excellent documentation while lacking the machine-readable metadata required for automated discovery and citation.

5.6 Baseline Comparison

This section compares the proposed hybrid assessment framework with two existing FAIR assessment approaches. The software-oriented howfairis tool is evaluated across all 30 ground truth repositories and provides a direct comparison of assessment scope. F-UJI is examined separately through a five-repository pilot to determine whether a data-oriented FAIR assessment tool can provide a meaningful quantitative baseline when applied directly to research software repositories hosted on GitHub.

5.6.1 F-UJI Baseline Applicability

Before using F-UJI as a quantitative baseline, its applicability to research software repositories was investigated. F-UJI is designed primarily for assessing the FAIRness of research data objects. The pilot therefore examined whether it could meaningfully differentiate research software repositories when provided with plain GitHub repository URLs.

Five ground truth repositories with different rule-based and documentation quality profiles were submitted directly to F-UJI (version 3.5.1, metrics v0.8): `astropy/astropy` and `sunpy/sunpy` with rule-based scores of 6/6, `gonum/gonum` with a rule-based score of 2/6 and an LLM documentation score of 5/12, `yt-project/yt` with scores of 4/6 and 11/12, respectively, and `scipy/scipy` with scores of 5/6 and 8/12. Table 5.12 presents the results.

Table 5.12: F-UJI assessment results compared with the proposed framework's rule-based and LLM documentation scores (n=5).

Repository	Rule	LLM	F-UJI	F	A	I	R
`astropy/astropy`	6/6	7/12	42.31%	42.86%	42.86%	33.33%	50.00%
`sunpy/sunpy`	6/6	9/12	42.31%	42.86%	42.86%	33.33%	50.00%
`gonum/gonum`	2/6	5/12	42.31%	42.86%	42.86%	33.33%	50.00%
`yt-project/yt`	4/6	11/12	42.31%	42.86%	42.86%	33.33%	50.00%
`scipy/scipy`	5/6	8/12	42.31%	42.86%	42.86%	33.33%	50.00%

Despite rule-based scores ranging from 2/6 to 6/6 and LLM documentation scores ranging from 5/12 to 11/12, all five repositories received the same F-UJI score of 42.31% (11/26) and the same FAIR category breakdown. F-UJI also reported that the evaluated resources did not identify themselves as datasets, with the detected resource types including `softwaresourcecode` and `object`.

Within this pilot, F-UJI therefore provided no differentiation between repositories with substantially different software-specific FAIRness profiles. Inspection of the metric-level outputs suggests that several F-UJI metrics rely on metadata representations that are not directly aligned with repository-level software evidence. For example, the license metric for `astropy/astropy` failed despite the repository explicitly identifying a BSD 3-clause license.

Given the identical results across the five varied repositories and F-UJI's resource-type warning, a quantitative comparison across all 30 ground truth repositories was not pursued. The pilot is instead treated as an applicability finding: when applied directly to the tested GitHub repository URLs, F-UJI was not a sufficiently discriminative quantitative baseline for the software-oriented assessment performed in this study.

5.6.2 howfairis Assessment Results

The howfairis tool was applied to all 30 ground truth repositories. howfairis evaluates five binary indicators: open repository, license, software registry, citation, and checklist (presence of a fair-software.eu badge). Table 5.13 summarises the score distribution.

Only one repository (`Ouranosinc/xclim`) achieved a full score of 5/5. The checklist indicator, which requires the presence of a fair-software.eu badge in the repository README, was satisfied by only one repository in the ground truth set. Con-

Table 5.13: howfairis score distribution across the 30 ground truth repositories.

Score	Count	Percentage
5/5	1	3%
4/5	15	50%
3/5	11	37%
2/5	3	10%

sequently, this indicator provided little differentiation between repositories in the evaluated sample. Both the open repository and license indicators passed for all or nearly all repositories, consistent with the rule-based results reported in Section 5.3.

5.6.3 Scope Comparison

Table 5.14 compares the assessment dimensions covered by howfairis, the rule-based component, and the LLM-based component of the proposed framework. F-UJI is not included in this dimension-by-dimension table because the pilot demonstrated that its data-oriented metrics were not directly comparable to the software-specific dimensions evaluated by the proposed framework.

Table 5.14: Comparison of assessment scope across howfairis, the rule-based component, and the LLM-based component.

Dimension	howfairis	Rule-based	LLM-based
Open repository	✓	–	–
License	✓	✓(A1)	–
Software registry	✓	–	–
Citation metadata	✓	✓(F2, R2)	–
Checklist badge	✓	–	–
Persistent identifier	–	✓(F1)	–
Dependency file	–	✓(I1)	–
Installation clarity	–	–	✓(D1)
Usage clarity	–	–	✓(D2)
Purpose description	–	–	✓(D3)
Dependency description	–	–	✓(D4)
Examples/tutorials	–	–	✓(D5)
Contribution info	–	–	✓(D6)

The comparison shows that howfairis and the proposed framework cover largely complementary assessment dimensions. Both approaches assess licensing and citation-related metadata, which represents the primary overlap between them. The proposed framework additionally evaluates persistent identifiers and dependency specifications through its rule-based component, as well as six documentation quality

dimensions through its LLM-based component. These aspects are not assessed by howfairis.

During the evaluation, the software registry indicator appeared to depend on repository badges displayed in the README rather than direct verification of registry entries. As a result, repositories available through a software registry but lacking the corresponding badge received a negative registry score.

5.7 Summary of Findings

This section summarises the key findings from the evaluation and addresses the research questions defined in Chapter 1.

5.7.1 Ground Truth Reliability

Inter-annotator agreement between the two human annotators was substantial for both the AI-based documentation quality dimensions (macro-average κ=0.709) and the rule-based indicators (macro-average κ=0.750, excluding the undefined A1 and R1 values). These results demonstrate sufficient reliability of the manual annotations to serve as ground truth for the subsequent evaluation. An initial fair agreement on I1 (dependency file, κ=0.286) was traced to an incomplete file-detection schema used during ground-truth collection rather than genuine annotator disagreement; following correction, I1 agreement improved to κ=0.634 (substantial).

5.7.2 Rule-Based Assessment

The rule-based scorer achieved exact agreement with the final ground truth annotations across all 30 repositories following iterative refinement. After excluding the 10 repositories used during the original pilot, precision, recall, and F1-score remained 1.000 across all six indicators on the remaining 20 repositories. This is treated as an internal consistency check rather than independent validation because some rule-based annotations in this subset were corrected during final scorer validation. Applied to the full 188-repository dataset, the scorer revealed substantial variability in structural FAIR compliance, with a mean score of 3.86 out of 6. README files and license files were near-universal (100% and 97% respectively), while dedicated software metadata files (31%) and persistent identifiers (51%) were considerably less common.

5.7.3 LLM-Based Assessment

LLM-human agreement was fair overall, with a macro-average weighted Kappa of 0.329 against the primary annotator and 0.275 against the second annotator. When the 10 pilot repositories were excluded, agreement with the primary annotator remained comparable and increased slightly from a macro-average weighted Kappa

of 0.329 to 0.373 on the 20-repository pilot-exclusion subset. The LLM systematically assigned lower scores than human annotators, most notably for D5 (examples and tutorials) and D2 (usage clarity). These dimensions are particularly sensitive to the README-only evaluation design because relevant usage material is often provided through external documentation links that were not followed by the LLM. The strongest agreement was observed for D1 (installation clarity, κ=0.605), where instructions are typically provided inline. The LLM did not reproduce manual assessments with the same level of consistency observed between the two human annotators.

5.7.4 Hybrid FAIRness Profile

Spearman's rank correlation between rule-based and LLM totals was weak and not statistically significant ($\rho = 0.175$, $p = 0.356$). No clear monotonic association was therefore detected in the evaluated sample. This result supports treating the two components as complementary assessment dimensions rather than combining them into a single score.

5.7.5 Baseline Comparison

The howfairis comparison showed that the proposed framework extends the scope of a software-oriented metadata assessment by adding six documentation quality dimensions concerned with installation, usage, purpose, dependencies, examples, and contribution guidance. The F-UJI pilot provided a different finding: all five tested GitHub repository URLs received the same FAIR score of 42.31% despite substantial differences in their rule-based and documentation quality profiles. Together with F-UJI's warning that the evaluated resources were not identified as datasets, this indicates limited applicability of F-UJI as a direct quantitative baseline for the software repositories evaluated in this study.

5.7.6 Summary

The results presented in this chapter provide the empirical basis for addressing the two research questions defined in Chapter 1. RQ1 concerns the extent to which LLM-based assessment complements rule-based FAIRness assessment; RQ2 concerns the reliability of LLM-based assessment and its failure modes. Both are addressed in detail in Chapter 6, drawing on the ground truth reliability, rule-based and LLM-based agreement results, hybrid profile analysis, and baseline comparison presented above.

6 Discussion

This chapter interprets the results presented in Chapter 5 in relation to the two research questions defined in Chapter 1, positions the proposed approach against related work, and reflects on the limitations and threats to validity of the study. The following sections address RQ1 and RQ2 in turn, before discussing the findings in relation to existing work, their practical implications, the limitations of the study, and directions for future work.

6.1 RQ1: LLM-Based Assessment as a Complement to Rule-Based Assessment

RQ1 asked to what extent LLM-based assessment can complement rule-based FAIRness assessment by capturing qualitative documentation characteristics of research software repositories. The results presented in Chapter 5 support a qualified but consistent answer: the LLM-based component adds genuinely new assessment coverage, but its value lies in what it evaluates rather than in how accurately it agrees with human judgement.

The clearest evidence for complementarity is structural. The rule-based and LLM-based totals showed only a weak, statistically non-significant association across the 30 ground truth repositories ($\rho = 0.175$, $p = 0.356$). The weak association is therefore consistent with the two components capturing largely distinct aspects of repository FAIRness, rather than providing redundant measurements of the same characteristics. The example profile presented in Section 4.6 illustrates this concretely: `astropy/astropy` satisfies all six rule-based indicators (6/6) while receiving one of the lower LLM-based documentation scores in the ground truth corpus (4/12). A purely rule-based assessment would indicate that the repository satisfies all six structural indicators evaluated in this work, while the LLM-based component identifies that a new user attempting to install and use the software from the README alone would find limited concrete guidance. This is precisely the kind of gap that motivated the hybrid design in Chapter 3: the presence of required files does not guarantee that those files, or the README itself, communicate what a user needs to reuse the software.

The baseline comparison in Chapter 5 provides converging evidence. The six documentation quality dimensions evaluated by the LLM-based component — installation clarity, usage clarity, purpose, dependencies, examples, and contribution guidance — are not represented at all in howfairis, which restricts its assessment to license, citation, registry, and checklist-style structural indicators. The F-UJI pilot

points in the same direction from a different angle: when applied to five repositories with substantially different rule-based and LLM-based profiles, F-UJI returned the same overall FAIRness score for all five. Within this small pilot, F-UJI therefore did not distinguish between repositories whose structural and documentation-based profiles differed substantially. This is consistent with its metadata-oriented design for research data objects and further illustrates that the documentation-quality dimensions examined in this thesis are not directly represented in its assessment.

This complementary coverage, however, does not imply that the LLM-based assessments are themselves highly accurate. LLM–human agreement was fair overall ($\kappa = 0.329$ against the primary annotator), substantially below the substantial agreement observed between the two human annotators ($\kappa = 0.709$). The LLM-based component should therefore be understood as extending the scope of automated FAIRness assessment to a dimension that rule-based tools cannot reach, rather than as a reliable substitute for either rule-based checks or human judgement within that dimension. The following section examines this reliability question, and the failure modes behind it, in detail.

6.2 RQ2: Reliability of LLM-Based Assessment and Its Failure Modes

RQ2 asked how reliably an LLM can assess qualitative documentation dimensions of research software repositories, and what failure modes affect its assessments. The results indicate that reliability is not uniform: it varies considerably across dimensions, and the observed disagreement reveals several systematic patterns rather than appearing entirely random.

Agreement with the primary annotator ranged from slight for some dimensions to moderate for others, with a macro-average weighted Kappa of 0.329 — fair by the Landis and Koch scale, and substantially below the 0.709 observed between the two human annotators. The strongest agreement was found for D1 (installation clarity, $\kappa = 0.605$), the weakest for D5 (examples and tutorials, as low as $\kappa = 0.156$ against the second annotator). This spread is not evenly distributed across dimensions at random: the two weakest-performing dimensions, D2 (usage) and D5 (examples), are precisely those most likely to be documented through external links rather than inline README content. Because the LLM-based component was restricted to README text and could not follow or evaluate linked resources, repositories that referred users to external tutorials, usage guides, or example galleries received lower scores from the LLM than from human annotators, who took the existence of those resources into account. This constitutes the most prominent and consistent failure mode identified in the qualitative error analysis. The pattern suggests that the restricted evidence available to the model contributed substantially to the observed underscoring: when relevant documentation was located outside the supplied README text, the model could not evaluate its content directly.

One further pattern illustrates the importance of interpreting Kappa alongside the underlying score distribution. D3 (purpose description) shows a negative Kappa ($\kappa = -0.071$) despite 86.7% exact agreement with the primary annotator. The human annotations are strongly concentrated in one category, with 28 of the 30 repositories receiving a score of 2. Under such low class variation, Kappa is sensitive to the prevalence of the dominant category and can produce a low or negative value despite high observed agreement. The D3 result should therefore not be interpreted from Kappa alone; the exact agreement and score distribution provide important additional context.

The model's self-reported confidence labels were also examined as a potential indicator of prediction reliability, with an ambiguous result. Confidence was not monotonically associated with accuracy: MEDIUM-confidence predictions achieved higher exact agreement than HIGH-confidence predictions overall, and this pattern held for several individual dimensions as well. A HIGH confidence label therefore cannot be interpreted as a guarantee of closer agreement with human judgement using the current prompting approach. This is a meaningful limitation for any downstream use of the confidence field as a triage signal, since it suggests the model can express high confidence in exactly the cases — typically involving the external-link failure mode described above — where its assessment diverges most from human interpretation of the rubric.

By contrast, no evidence was found for a systematic verbosity bias: the correlation between README length and the difference between LLM and human scores was weak and not statistically significant ($\rho = -0.169$, $p = 0.371$). The direction of the correlation for the dimensions most affected by underscoring (D2 and D5) was, if anything, opposite to what a simple verbosity-bias explanation would predict. README length therefore does not appear to explain the observed disagreement pattern in this sample, whereas the qualitative error analysis provides stronger evidence for limitations associated with externally located documentation.

Taken together, these results suggest that LLM-based assessment in this setting is more reliable for documentation characteristics that are stated explicitly and directly within the README, and less reliable when the relevant evidence is partial, located outside the supplied text, or requires resolving ambiguity that a structured rubric alone does not fully specify. The LLM should accordingly be regarded as a semantic assessor whose reliability is evidence-dependent rather than uniform, best used alongside deterministic structural checks and with its confidence labels treated as contextual information rather than a validated reliability estimate.

6.3 Comparison with Related Work

The results of this thesis can be situated against two strands of related work introduced in Chapter 2: automated LLM-based FAIR assessment, represented most directly by Fair-Way, and the broader LLM-as-judge literature on using language models as automated evaluators.

Fair-Way [9] is the closest related system, applying LLMs to automate FAIR assessment against the FAIRsFAIR DOAM metrics for research datasets. Its authors report that GPT-4o, the strongest model evaluated, reaches an exact match accuracy of 0.905 against their benchmark, with smaller and open-source models performing considerably worse. The LLM–human agreement observed in this thesis ($\kappa = 0.329$ against the primary annotator) is not directly comparable to Fair-Way's exact match figures, since the two studies evaluate different constructs (FAIR-related metadata correctness versus README documentation quality against an ordinal rubric) using different metrics and different underlying models. Nevertheless, Fair-Way's model comparison demonstrates that model choice can substantially affect LLM-based FAIR assessment performance. The present study evaluates only a single locally hosted 8-billion parameter model and therefore cannot determine how strongly the observed reliability depends on model choice. Evaluating larger or alternative models on the same documentation-assessment task remains necessary before conclusions about model-dependent performance can be drawn.

More substantively, this thesis differs from Fair-Way in scope and architecture rather than only in performance. Fair-Way targets research datasets and evaluates structured and semi-structured metadata using a purely LLM-based pipeline; the authors explicitly identify hybrid approaches combining LLMs with rule-based verification as a direction for future work, motivated by the hallucination risk of a purely LLM-based design. This thesis addresses that direction directly, targeting research software repositories rather than datasets, and combining a deterministic rule-based component with an LLM-based component that is scoped specifically to evaluate unstructured README documentation rather than structured metadata records. The complementarity finding discussed in Section 6.1 — that the rule-based and LLM-based components capture largely distinct information — provides empirical support for the value of this hybrid design in the software domain, extending the direction Fair-Way's authors proposed but did not evaluate.

The results also relate to the broader LLM-as-judge literature. Zheng et al. [12] found that a strong, closed-source model (GPT-4) can reach agreement with human raters approaching human–human levels in certain open-ended evaluation settings, while also identifying biases such as position bias, verbosity bias, and self-enhancement bias as recurring sources of unreliability. The present results are broadly consistent with the more cautious reading of that finding: agreement in this thesis falls well short of human–human levels, but the specific biases examined — verbosity bias in particular — were not observed in this sample, and structured rubric-based scoring was used throughout, consistent with the mitigation Zheng et al. suggest for improving scoring reliability. The failure mode identified in this thesis, systematic underscoring driven by an evidence-restriction policy rather than a length- or position-based bias, is not one of the biases directly characterised by Zheng et al., and represents a task-specific finding rather than a general property of LLM-as-judge evaluation reported elsewhere in the literature.

Overall, this comparison shows that the reliability limitations observed in this thesis are compatible with broader findings on the limitations of LLM-based eval-

uation. At the same time, the proposed approach explores the hybrid rule-based and LLM-based direction identified by Fair-Way in the specific context of research software documentation.

6.4 Implications

The findings of this thesis carry several practical implications for how a hybrid FAIRness assessment framework of this kind should be used and interpreted, beyond the specific numerical results reported in Chapter 5.

The most direct implication concerns how the two assessment components should be consumed together. Because the rule-based and LLM-based scores were shown to capture largely distinct information (Section 6.1), a hybrid profile is most useful when read as two separate signals rather than as a single quality indicator. A repository with a high rule-based score should not be assumed to have well-documented, easily reusable software, and a repository with a low rule-based score is not necessarily poorly documented. The example presented in Section 4.6 makes this concrete: a repository can satisfy every structural indicator evaluated in this work while still leaving a new user with limited concrete guidance on how to install and use the software. Presenting the two dimensions separately, rather than combining them into an overall FAIRness score, allows this kind of gap to remain visible rather than being averaged away.

A second implication concerns how the LLM-based scores themselves should be treated. Given a macro-average agreement of $\kappa = 0.329$ with the primary annotator, the LLM-based dimension scores are best interpreted as an approximate, assistive signal rather than as an authoritative substitute for human review. This is particularly relevant for D2 and D5, which showed some of the clearest disagreement patterns associated with documentation located outside the README. A practical consequence is that low LLM scores on these two dimensions should be treated as an indication that manual inspection may be warranted, rather than as a reliable judgement that the corresponding documentation is genuinely absent or inadequate.

A related implication concerns the confidence labels produced alongside each score. Because HIGH-confidence predictions did not consistently outperform MEDIUM-confidence predictions, the confidence field cannot currently be used as a validated reliability estimate for prioritising which assessments deserve human review. Until confidence calibration is demonstrated empirically, or a more reliable uncertainty signal is developed, the confidence label is better treated as contextual information accompanying a score — of potential interest when reviewing an individual repository's profile — rather than as a mechanism for automatically triaging large numbers of repositories by predicted reliability.

Finally, the baseline comparison suggests a practical positioning for the proposed approach relative to existing tools rather than a claim of outright superiority. Where howfairis provides fast, purely structural checks and F-UJI is designed for research data objects rather than software repositories, the LLM-based component of this

framework adds a distinct assessment layer — documentation quality as experienced by a prospective user — that is not directly represented by either tool in the comparisons conducted in this thesis. This suggests the two kinds of tools are better viewed as complementary than as competitors: an organisation or repository maintainer seeking a fast, deterministic compliance check retains good reasons to use a tool such as howfairis, while the qualitative documentation assessment introduced in this thesis addresses a different, additional question that such tools do not answer.

6.5 Limitations and Threats to Validity

This section discusses the limitations of the study and the threats to validity that should be considered when interpreting its results.

Single-model scope. The LLM-based component was evaluated using a single, locally hosted 8-billion parameter model (Llama 3.1 8B) at a fixed temperature. As discussed in Section 6.3, the reliability limitations observed in this thesis cannot be disentangled from this specific model choice: it remains unknown whether a larger or closed-source model would substantially improve agreement with human annotators, reduce the systematic underscoring pattern identified in Section 6.2, or produce better-calibrated confidence labels. The findings of this thesis should therefore be interpreted as characterising the evaluated model and assessment setup, rather than as a model-independent property of LLM-based FAIRness assessment.

Input truncation strategy. README content exceeding 12,000 characters was truncated using a fixed sequential cutoff rather than a token-based or section-aware strategy, as described in Chapter 4. Within the 30-repository ground truth corpus, only one README exceeded this threshold, and manual inspection of the truncated portion showed no loss of relevant D1–D6 evidence in that particular case. This single instance does not establish that fixed character-based truncation is safe in general; a section-aware strategy that preserves installation, usage, dependency, example, and contribution content specifically would be more robust for longer or differently structured READMEs than the corpus evaluated here.

Representativeness of the ground truth corpus. The mean rule-based score across the 30-repository ground truth corpus (4.30 out of 6) was somewhat higher than the mean across the full 188-repository dataset (3.86 out of 6). Because the ground truth corpus was constructed to maximise diversity across research domains and programming-language ecosystems rather than sampled at random from the full dataset, it may not be fully representative of the broader population of repositories to which the rule-based scorer was subsequently applied. Agreement and validation results reported against the ground truth corpus should accordingly be interpreted as characterising performance on a deliberately diverse but non-random sample, rather than as an unbiased estimate of performance across the full dataset.

Partial exclusion logging. During dataset construction, the keyword-based exclusion step was logged explicitly, recording the specific repositories removed for containing predefined non-software-related keywords. Repositories excluded for be-

ing forks, archived, or inactive since before the collection cutoff date were filtered during the initial collection stage but were not separately logged. Consequently, a complete, itemised record of every excluded repository and its specific exclusion reason is not available for the full 198-to-188 filtering process, limiting the transparency of this step relative to the ideal of a fully auditable exclusion trail.

Ground truth correction process. Corrections made to the ground truth annotations during validation — including the dependency-file schema correction and the license-annotation correction for `rust-ml/linfa` discussed in Chapter 5 — were applied directly to the annotation records rather than being tracked in a separate change log. While each correction was traceable to a specific, documented cause and was applied consistently using the finalised annotation guidelines, the absence of a separate audit trail means that the sequence of corrections cannot be independently reconstructed from the released annotation files alone.

Scale of the ground truth corpus. With 30 manually annotated repositories, the ground truth corpus is sufficient to support the agreement and validation analyses reported in this thesis, but Kappa and correlation estimates computed on a sample of this size carry non-trivial uncertainty, as illustrated by the prevalence-effect result observed for D3 in Section 6.2. A larger annotated corpus would allow for more precise estimates and a more fine-grained analysis of agreement across research domains and programming languages.

README-only evidence scope. The LLM-based assessment was restricted to the README content supplied to the model and did not retrieve or inspect documentation available through external links. This restriction ensured that every repository was evaluated against a consistent and reproducible evidence source, but it also limits the construct being measured. In particular, repositories that place usage guides, tutorials, example galleries, or other documentation outside the README may receive lower scores even when such resources are available to users. The qualitative error analysis in Section 6.2 indicates that this limitation particularly affected D2 and D5. The reported LLM scores should therefore be interpreted as assessments of documentation available directly from the README, rather than of the complete documentation ecosystem of a repository.

Subjectivity of qualitative ground truth. The documentation dimensions D1–D6 require human judgement about the completeness and clarity of repository documentation. Although the substantial agreement between the two human annotators ($\kappa = 0.709$) provides evidence that the rubric can be applied with reasonable consistency, disagreement remained for individual dimensions and repositories. The manual annotations should therefore not be treated as an error-free absolute ground truth, but as a reference assessment produced according to the operationalised rubric defined in Chapter 3. This uncertainty should be considered when interpreting LLM–human disagreement, since some differences may reflect ambiguity in the assessment task itself rather than model error alone.

Scope of the baseline comparison. The comparison with existing FAIR assessment tools was limited in scope. While howfairis was applied to the ground truth repositories, the F-UJI analysis was conducted as a five-repository pilot be-

cause the tool is primarily designed for research data objects rather than GitHub software repositories. The identical F-UJI scores observed in this pilot are therefore illustrative of the behaviour encountered for the selected repositories and should not be interpreted as a comprehensive benchmark of F-UJI against the proposed framework.

6.6 Future Work

The findings and limitations discussed in this chapter point to several directions for extending the proposed framework.

Evaluating additional LLMs. As discussed in Sections 6.3 and 6.5, the reliability results reported in this thesis are specific to a single, locally hosted 8-billion parameter model. A natural extension would be to apply the same rubric, prompt structure, and evaluation procedure to additional open-source and proprietary models of different sizes and capabilities. Such a comparison would help determine whether the observed agreement levels and failure modes are primarily associated with the assessment task or with the particular model configuration evaluated in this work.

Section-aware and token-based input handling. The current implementation applies a fixed character-based truncation cutoff to README content, as described in Chapter 4. Future work could implement a section-aware preprocessing strategy that identifies and preserves content relevant to each documentation dimension — installation, usage, dependencies, examples, and contribution guidance — regardless of its position in the README, together with a token-based length limit that more directly reflects the model's actual context capacity.

Addressing the external-link limitation. Since one of the most prominent failure modes identified in this thesis is associated with the LLM's inability to evaluate documentation located outside the README, future work could investigate retrieving and incorporating linked documentation — such as dedicated usage guides or example galleries — into the assessment evidence, provided this can be done without substantially increasing assessment cost or compromising reproducibility.

Repeated-run stability analysis. The current evaluation reports scores from a single inference run per repository. Running the LLM-based assessment repeatedly for the same repositories would allow score consistency and dimension-level variation across runs to be measured directly, complementing the confidence-calibration analysis presented in Chapter 5.

Scaling and diversifying the ground truth corpus. Expanding the manually annotated corpus beyond 30 repositories, with a sampling strategy closer to the composition of the full dataset, would reduce the estimation uncertainty discussed in Section 6.5 and allow agreement and validation results to be reported with greater confidence, including analysis broken down by research domain and programming-language ecosystem.

Pipeline integration and user-facing presentation. The current implemen-

tation consists of independently executable components, as described in Chapter 4. Future work could integrate these components behind a single command-line interface and package the framework for straightforward execution against a GitHub repository. A lightweight web interface could additionally present the resulting hybrid profile visually, showing structural indicators and documentation-quality dimensions separately together with the supporting reasoning. This would make the distinction between the two assessment components more accessible to repository maintainers without altering the underlying scoring methodology.

Improved uncertainty and review mechanisms. The confidence analysis showed that the model's self-reported confidence labels do not provide a calibrated estimate of assessment reliability. Future work could therefore investigate alternative uncertainty measures, including consistency across repeated inference runs or disagreement between multiple models. Such signals could support a review mechanism in which uncertain or conflicting assessments are flagged for human inspection while deterministic structural indicators remain fully automated.

7 Conclusion

This thesis addressed the problem of automated FAIRness assessment for research software repositories, motivated by the observation that existing tools evaluate the presence of structured metadata and repository files but cannot assess the quality and completeness of unstructured documentation. To address this gap, a hybrid assessment framework was designed, implemented, and evaluated, combining a deterministic rule-based component covering six FAIR4RS-derived structural indicators with an LLM-based component evaluating six documentation-quality dimensions of repository README content.

The framework was applied to a dataset of 188 research software repositories collected from GitHub across ten research domains, with a manually annotated ground truth corpus of 30 repositories constructed to validate both assessment components. The rule-based component was shown to reproduce its operational definitions faithfully against the ground truth and revealed substantial variability in structural FAIR compliance across the full dataset. The LLM-based component, evaluated using a locally hosted Llama 3.1 8B model, achieved fair agreement with human annotators overall ($\kappa = 0.329$), with reliability that varied considerably across dimensions and was closely tied to whether the relevant evidence was present directly in the README or located externally.

In relation to the two research questions guiding this work, the results indicate that LLM-based assessment complements rule-based FAIRness assessment: the two components were shown to capture largely distinct aspects of repository FAIRness, with only a weak and non-significant association between their scores, and the LLM-based component evaluates documentation-quality dimensions that existing tools such as howfairis and F-UJI do not represent (RQ1). At the same time, the reliability of LLM-based assessment is evidence-dependent rather than uniform: it performs best on documentation characteristics expressed explicitly within the README and is systematically less reliable when relevant information is located externally, while the model's self-reported confidence labels were found not to be reliably calibrated with respect to this reliability (RQ2).

The primary contribution of this thesis is a working hybrid FAIRness assessment framework for research software repositories that preserves structural and documentation-based evidence as separate, interpretable dimensions rather than collapsing them into a single score, together with an empirical evaluation — against manual ground truth, human inter-annotator agreement, and two existing baseline tools — that characterises both the added assessment coverage and the reliability limitations of the LLM-based component. A secondary contribution is the identification of a specific failure pattern in LLM-based documentation assessment: sys-

tematic underscoring associated with documentation evidence located outside the README, together with the finding that self-reported model confidence does not currently provide a calibrated reliability signal. The analysis found no evidence of a systematic relationship between README length and the observed scoring disagreement.

Taken together, this work demonstrates that combining rule-based and LLM-based assessment can broaden the scope of automated FAIRness evaluation for research software by bringing structural and documentation-based evidence into a common assessment framework, while also showing that the reliability of the LLM-based component, and the conditions under which it can be trusted, must be established empirically rather than assumed. The framework developed in this thesis, and the limitations identified in evaluating it, provide a concrete basis for the future work outlined in Chapter 6.

Bibliography

[1] Barker, M., et al.: Introducing the FAIR principles for research software. Scientific Data 9(622) (2022)

[2] Chue Hong, N., Breitmoser, E., et al.: D5.2 - metrics for automated FAIR software assessment in a disciplinary context. Tech. rep., FAIR-IMPACT Project (2025), v1.1

[3] Dvivedi, S.S., Vijay, V., Pujari, S.L.R., Lodh, S., Kumar, D.: A comparative analysis of large language models for code documentation generation. In: Proceedings of the 1st ACM International Conference on AIware (AIware 2024) (2024)

[4] FAIRsFAIR: F-UJI automated FAIR data assessment tool (2021), `https://www.f-uji.net`

[5] Hou, X., et al.: Large language models for software engineering: A systematic literature review. ACM Transactions on Software Engineering and Methodology (2024)

[6] Imani, A., Radmanesh, S., Ahmed, I., Moshirpour, M.: Does documentation matter? an empirical study of practitioners' perspective on open-source software adoption. arXiv preprint arXiv:2403.03819 (2024)

[7] Landis, J.R., Koch, G.G.: The measurement of observer agreement for categorical data. Biometrics 33(1), 159–174 (1977)

[8] Prana, G.A.A., Treude, C., Thung, F., Atapattu, T., Lo, D.: Categorizing the content of GitHub README files. vol. 24, pp. 1296–1327 (2019)

[9] Sharma, A., Sowe, S.K., Kim, S.Y., Hoseini, S., Limani, F., Boukhers, Z., Lange, C., Decker, S.: FAIR data assessment using LLMs: The fair-way. In: Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25) (2025)

[10] Treude, C., Middleton, J., Atapattu, T.: Beyond accuracy: Assessing software documentation quality. In: Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20). pp. 1509–1512 (2020)

[11] Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., et al.: The FAIR guiding principles for scientific data management and stewardship. Scientific Data 3, 160018 (2016)

[12] Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I.: Judging LLM-as-a-judge with MT-bench and chatbot arena. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)